Debreceni Egyetem
Informatikai Kar
Adattudomány és Vizualizáció Tanszék

Dinamikus objektumelkerülés megerősítéses tanulással a CARLA szimulátorban

Témavezető:
Mészáros László
Tanársegéd

Készítette:
Angi Dávid
Adattudomány MSc

Debrecen, 2026

Tartalomjegyzék

1. Bevezetés **4**

2. Szimulációs környezet **5**

3. Programozás és algoritmus **7**

3.1. Megerősítéses tanulás 7

3.2. Python programozási nyelv 9

3.3. Soft Actor-Critic és a Stable-Baselines3 10

4. Szenzoradatok előfeldozása, Autóenkóderek **11**

4.1. Kamera 12

4.1.1. Konvolúciós autoenkóder 13

4.1.2. Reziduális autoenkóder 14

4.1.3. VGG-alapú autoenkóder 14

4.1.4. Összehasonlítás és a választott modell 15

4.2. LiDAR 16

4.2.1. BEV-alapú autoenkóder 18

4.2.2. Gráf-alapú autoenkóder 19

4.2.3. Point-MAE 20

4.2.4. Összehasonlítás és a választott modell 21

4.3. Szenzorfuzio 22

5. Jutalom függvény **24**

5.1. R1: sávtartás és szabályos előzés 25

5.2. R2: ütközésidő-büntetés előzhető helyzetben 26

5.3. R3: hangolás bátrabb előzésre 26

5.4. R4: haladásalapú jutalom 27

5.5. R5: potenciálalapú jutalomformálás 28

6. Elkészült modellek összehasonlítása **28**

6.1. A kiértékelés módszertana 28

6.2. A vizsgált jutalomváltozatok 30

6.3. Eredmények 30

6.4. A módosítások értékelése 32

6.5. Az eredmények megbízhatósága 33

7. Összegzés **33**

Hivatkozások **34**

1. Bevezetés

A 21. század egyik ha nem a legnagyobb hatású technológiai átalakulása a közlekedésben zajlik. Az elmúlt másfél évtizedben szemünk láttára kerültek önvezető járművek a kutatólaboratóriumokból a hétköznapi forgalom részévé. Kezdetben vezetéstámogató rendszerek (sávtartó automatika, adaptív tempomat, vészfékasszisztens) kirivó már technológiának számítottak azonban ma már a középkategóriás autókban is alapfelszereltség részét képezik, egyes városokban pedig sofőr nélküli robottaxik szállítják az utasokat. A fejlődés robbanásszerű gyorsulását a szenzorok relatíve olcsóbbá válása, valamint a járművekbe építhető, nagy teljesítményű számítási egységek megjelenése nélkülözhetetlen volt. Ezen felül az adatfeldolgozás és a mélytanulás fejlődése is kulcsszerepet ezen fejlődés történetben mivel, ezek a technológiák teszik lehetővé, a járművek számára környezetük érzékelését.

Az önvezető autók iránti érdeklődést elsősorban a biztonság motiválja. A közúti balesetekben világszerte évente több mint egymillió ember veszti életét [1], és a balesetek döntő többsége emberi hibára – figyelmetlenségre, fáradtságra, rossz helyzetmegítélésre – vezethető vissza.

Egy gép nem fárad el, semmi sem vonja el a figyelmét, és valamint a reakcióideje optimális esetben rövidebb lehet az emberinél. Ez az eset azonban csak akkor valósulhat meg, ha a vezetést végző algoritmus minden helyzetben megbízhatóan, hatékonyan és biztonságosan működik. Nem elég, ha a jármű az esetek túlnyomó részében – egyenes úton, jó időben, kiszámítható hanem nehézkes körülmények között is helyt kell állnia. Ezen oknál fogva avalódi kihívást a ritka, váratlan helyzetek jelentik, például a forgalom dinamikája, egy sávba kerülő akadály vagy egy előzés közben megváltozó forgalmi helyzet. Egyetlen rosszul kezelt helyzet is súlyos balesethez vezethet. (észlelés, predikció, tervezés, szabályozás) bontva oldják meg. Ezek a rendszerek jól értelmezhetők, de minden szabályt és határesetet előre meg kell fogalmazni bennük, ami a valós forgalom sokszínűsége mellett rendkívül nehéz. Alternatívát a megerősítéses tanulás (*reinforcement learning*, RL) kínál, amelyben az ágens nem előre megírt szabályokat követ, hanem a környezettel való interakció során, jutalmak és büntetések alapján maga tanulja meg, hogyan kell cselekednie. Ez a megközelítés különösen alkalmas az olyan döntési feladatokra, mint a sávtartás, az akadályok kikerülése vagy az előzés, ahol a helyes viselkedést nehéz szabályokkal leírni, de könnyű megmondani, hogy egy adott kimenetel jó vagy rossz.

A vezetési feladatot hagyományosan egymásra épülő, alaposan tervezett modulokra bontják. Ezen blokkkat jól megtervezett logikai adatfolyam köti össze amelyet robotikai rendszerek kötnek össze (pl ROS2)[2]. Ilyen modulokra épülő önevezető rendszer például az Autoware Auto [3] amely egy ROS2 alapokra épülő robotikai rendszer önevezető autózáshoz. A rendszer kifinomultsága abban rejlik hogy minden felhasználó saját igényei szerint tudja formálni a rendszert és saját modulokat is adhat hozzá. Az ilyen bonyolult rendszerekkel szemben jelent egy lehetséges alternatívát a megerősítéses tanulás azonban ennek ára van. Az ágens próbálkozások és hibák sorozatán keresztül tanul amelyek költsége igen magas lenne a való világban, mivel

az elfogadható vezetési stratégia alakítása akár több százezer vagy akár több millió lépésen át is történhet. A tanulási folyamat során az ágensnek gyakorolnia kell és felfedeznie saját világát valamint korlátait. Ezt egy valós járművel elvégezni gyakorlatilag kivitelezhetetlen. Egy szenzorokkal felszerelt tesztjármű, a zárt tesztpálya, a biztonsági sofőr és a karbantartás költsége rendkívül magas, egy ütközés pedig nemcsak anyagi kárt okoz, hanem emberi életet is veszélyeztethet. Ráadásul az ágnest szeretnénk felkészíteni a váratlan és veszélyes helyzetekre, hogy azokat is megfelelőképp kezelje és elkerülje a baleseteket.

Ezen okoknál fogva a tanítást és a tesztelést szimulált környezetben végeztem. A szimulációra a CARLA-t [4] egy nyílt forráskódú, kifejezetten önvezetés-kutatásra fejlesztett szimulátor,amely valósághű városi és autópálya-környezetet is képes egyaránt szimulálni valamint a fizikai járműmodellt, amelyek elengedhetetlenek a reális valósághű szimulációkhoz.

Egy önvezető rendszer döntései csak annyira lehetnek jók, amennyire pontos a környezetéről alkotott képe, ez a kép pedig kizárólag a gépi látás és érzékelés révén a szenzorokból származik. A szenzor adatok feldolgozása az ágnes számára igazi kihivást jelent. A komplex szenzorokból sokszínű és nagy varianciájú adat érkezik. Ilyen elgondolás okán a munkám során különböző szenzor adat előfeldozásokat végeztem ezzel optimalizálva a feladatokat az ágens számára.

Munkám során olyan ágens kifejlesztésén szeretnék dolgozni amely képes forgalmi szituációkat(pl előzés) is megfelelőképp biztonságosan kezelni, valamint betartja a KRESZ előírásokat is egyaránt. A fő gondolatot **terapaptommakol2022obstacle** munkája adta, akik a CARLA szimulátorban DQN alapú ágenst tanítottak, amely a LiDAR pontfelhő, a pozíció és a sebesség alapján fékezéssel és kitéréssel sikeresen elkerülte az úton parkoló járművet. Ez a munka azonban kizárólag statikus akadályra terjedt ki. Erre építkezve célom a dinamikus objektumok elkerülése, vagyis hogy az ágens a mozgó, lassabban haladó járműveket is biztonságosan, a KRESZ szabályait betartva kerülje ki.

2. Szimulációs környezet

A tanításhoz és a teszteléshez a CARLA (*Car Learning to Act*) szimulátorra [4, 5] esett a választásom. (1. ábra) A CARLA egy nyílt forráskódú UnrealEngine4 alapú szimulációs környezet, amelyet kifejezetten önvezető rendszerek fejlesztésére, tanítására, validálására és tesztelésére hoztak létre. A szimulátor magja C++ nyelven készült, a megjelenítést és a fizikai szimulációt pedig az Unreal Engine 4 (UE4) játékmotor végzi. Ennek köszönhetően a szimulátor valósághű, képet és járműdinamikát nyújt. A program képes a valóságban is gyakran használt önvezető szenzorok – többek között RGB-kamera, mélységkamera, szemantikus szegmentációs kamera, LiDAR, radar, GNSS és IMU – szimulált megfelelőit generálni minden forgalmi szituációban. Továbbá fel van készítve a dinamikus időjárás, napszak realisztikus kezelésére is. A forgalom pedig szabadon állítható és szabályozható. Így az ágens olyan típusú szenzoradatokból tanulhat, mint amilyeneket egy valódi jármű is szolgáltatna.

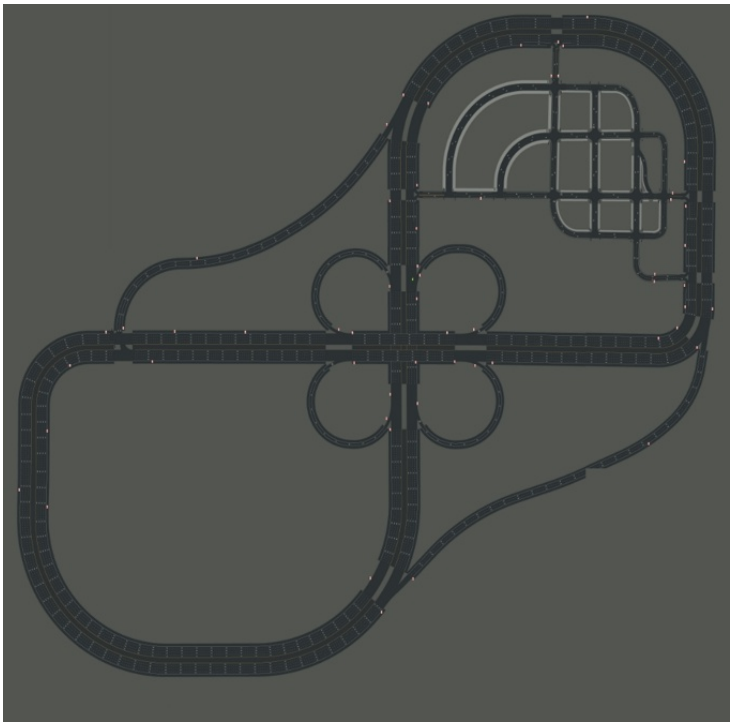


1. ábra. A CARLA szimulátor (forrás: [5])

A szimulátor kliens–szerver architektúrára épül [6], tehát a szerver futtatja magát a szimulációs környezetet ez által ő felel a világ és a járművek fizikájának számításáért, valamint a szenzoradatok előállításáért. Ez a kialakítás ugyanakkor teret enged annak is hogy a munka akár 2 számítógépen is megvalósuljon, például ez akkor indokolt ha nem elegendő egy számítógép grafikus feldolgozási erőforrása arra, hogy a szimuláció és az ágens tanulása is fusson egyaránt. A kliens program, melyet a felhasználó ír, hálózati interfészen (TCP kommunikációs protokol segítségével alapértelmezetten a 2000-es porton) keresztül kérdezi le a szimuláció állapotát, illetve küld parancsokat a szervernek. Ilyenek lehetnek például a jármű kormány parancsa, gáz adagolása és fékzés is, ugyanakkor a kliens így kapja meg az egyes szimulálni kívánt szenzoradatokat is, a megfigyelni kívánt járműről. A kliensoldali API Pythonban is elérhető, ami nagyban megkönnyíti a fejlesztést. Munkám során a CARLA 0.9.16-os verzióját használtam szinkron módban, rögzített (20 Hz-es) szimulációs lépésközzel. Szinkron módban a szerver csak akkor lép a következő állapotba, amikor a kliens erre utasítja ez nagy segítségemre volt az ágens egyes lépéseinél, így a szenzoradatok és az ágens döntései mindig ugyanahhoz a szimulációs lépéshez tartoztak, valamint a tanítás is reprodukálható marad.

A CARLA-ban adott a lehetőség útvonaltervezésre is. A pálya úthálózata útpontokból (*waypoint*) álló topológiai gráfként érhető el, amelyen a szimulátorhoz tartozó `GlobalRoutePlanner` segítségével két tetszőleges pont között játszi könnyedséggel tervezhető [6] bármilyen kívánt útvonal. Ezt kihasználva minden epizód elején véletlenszerűen választott kezdő- és célpont között új, dinamikus útvonalat tervezek, amelyen az ágensnek végig kell haladnia ezzel kiküszöbölve azt, hogy esetlegesen rátanuljon az adott útvonalra, így segítem az általánosítási képességét.

A szimulátor alapértelmezetten tartalmaz több előre elkészített pályát (*Town*), amelyek varianciája az átlagos önevezető feladatok többségét kielégíti. Munkám során ezek közül a



2. ábra. A Town04 pálya felülnézeti térképe (forrás: [6])

Town04-et választottam (2. ábra). Választásom erre a pályára esett mivel az itt kialakított kisváros tökéletesen megfelel az elvárásaimnak. Alakja „8-as” alakú, végtelenített autópályával [6] megfelelő célom eléréséhez. A pálya egyszerre tartalmaz városi utcákat és többsávos autópályát, fel- és lehajtókkal, valamint lóhere alakú csomóponttal. Amint említettem célom az autópályás előzés megtanítása, ez a pálya a legalkalmasabb. A többsávos autópályán az ágens a lassabban haladó járművet a bal oldali sávba kihúzódva szabályosan meg tudja előzni, a körbeérő autópálya pedig hosszú, megszakítás nélküli vezetési szakaszokat tesz lehetővé.

3. Programozás és algoritmus

A következőkben munkám elméleti és technikai hátterét írom le röviden. Elsőként a megerősítéses tanulás alapjai majd, ezt követően a fejlesztéshez használt programozási nyelvet és könyvtárait, végül a Soft Actor-Critic algoritmust és a Stable-Baselines3 keretrendszert. Mindegyiknél kitérve arra, miért esett az adott technológiára a választásom.

3.1. Megerősítéses tanulás

A gépi tanulás módszereit hagyományosan három nagy csoportba soroljuk. A *felügyelt tanulás* során a modell tisztában az elvárt kimenet valamint a bementi adatokkal egyaránt így tanul, vagyis minden tanítópéldához ismert a helyes válasz. Ide tartoznak a klasszikus osztályozási és regressziós feladatok, például képeken látható objektumok felismerése, de maga a vezetés is

megtanulható így. A *viselkedésklónozás* (*behavior cloning*) során a modell egy emberi sofőr rögzített vezetéséből, a szenzoradatok és a hozzájuk tartozó kormány valamint gázparancsok párjaiból tanulja meg utánozni a sofőrt. Ezt az elvet már az 1980-as évek végén alkalmazta az ALVINN rendszer [7]. A módszer korlátja, hogy a modell legfeljebb olyan jó lehet, mint a tanítóadat. Értem ez alatt, hogy a modell nagy varianciájú adaton kell tanuljon különben a váratlan helyzeteket nem tudja megfelelőképp kezelni. Általánosítási képessége kevésbé erős mint a többi gépitanúási megoldásnak. Következő módszer *felügyelet nélküli tanulás* ahol nincsenek címkék, a tanítás célja az adatok rejtett szerkezetének, például csoportjainak vagy tömör reprezentációjának feltárása. Tipikus példája a klaszterezés (például a k-közép módszer), amely a hasonló adatpontokat csoportokba rendezi.

A *megerősítéses tanulás*(*reinforcement learning*, RL) ezektől alapvetően lényegesen eltér. A tanuló ágensnek senki sem mondja meg, mi lett volna a helyes döntés egy adott helyzetben vagy mi lett volna az elvárt viselkedés, csupán egy számszerű visszajelzést, *jutalmat* kap viselkedéséről [8]. A tanulási folyamatnak két szereplője van egyik az *ágens*, a döntések meghozataláért felelős a *környezet*, amelyben az ágens mozog a folyamat során. Ezen páros kölcsoönhatása egymással diszkrét időlépésekben történik: az ágens minden lépésben megfigyeli a környezet aktuális *állapotát*, ez alapján végrehajt egy *akciót*, amelynek hatására a környezet új állapotba kerül, az ágens pedig jutalmat kap előző döntései alapján.A feladatot Markov-döntési folyamatként (*Markov Decision Process*, MDP) írhatjuk fel [8], amiben az állapotok, az akciók, és az állapotátmenet-valószínűségek valamint a jutalomfüggvény határoz meg. A fentiek alapján a modell feltevése a *Markov-tulajdonság*. A következő állapot csak a jelenlegi állapottól és akciótól függ. Mivel az ágens a világot szenzorain keresztül érzékeli, ezt tekintjük állapotnak.

Az ágens viselkedését a *stratégia* (*policy*) írja le, célja minden tanulási epizódban a *hozam* (*jutalom*) várható értékének maximalizálása:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \tag{1}$$

ahol r_{t+1} a t-edik lépés utáni jutalom, a $\gamma \in [0,1)$ diszkontfaktor pedig a jövőbeli jutalmak súlyát szabályozza. A tanulás *epizódokra* tagolódik, amelyek nálam ütközéssel vagy az útvonal teljesítésével érnek véget.

Esetemben nagy valamint, folytonos állapottérben (kamerakép, LiDAR) az értékfüggvényt és a stratégiát mély neurális hálóval közelítem (*deep reinforcement learning*). Ezen típusu algoritmusok között az áttörést a DQN hozta [9], amely *visszajátszási pufferrel* (*replay buffer*) és *célhálóval* (*target network*) stabilizálta a tanulást.

Az algoritmusválasztás szempontjából a fő különbségek [8]:

- **Modellalapú vs. modellmentes:** a CARLA fizikájának megtanulása nehéz lenne, ezért modellmentes módszert választottam.

- **Értékalapú, stratégiaalapú, aktor–kritikus:** utóbbiban az *aktor* a stratégia, a *kritikus* pedig értékeli a döntéseit.
- **On-policy vs. off-policy:** az on-policy módszerek (pl. PPO [10]) eldobják a régi tapasztalatot, az off-policy módszerek (DDPG [11], TD3, SAC [12]) újrahasznosítják, így kevesebb interakcióból tanulnak.
- **Diszkrét vs. folytonos akciótér:** a DQN csak véges akciók közül választhat.

3.2. Python programozási nyelv

A kísérletem implementálására a Python programozási nyelvet választottam ami, magas szintű, interpretált, dinamikusan típusos valamint általános célú. Megálmodója Guido van Rossum a nylev első változata 1991-ben jelentette meg [13]. Tervezési filozófiájának középpontjában az olvashatóság áll. Ezen elvre kiváló példa a kódblokkokat behúzás jelöli, a szintaxis tömör és közel áll a természetes nyelvhez, így a programok gyorsan megírhatók és könnyen érthetők. Ezen tulajdonságainak köszönheti, hogy ma a világ egyik legszélesebb körben használt programozási nyelve, a gépi tanulás és az adattudomány területén pedig gyakorlatilag szabványnak számít.

Elsőre ellentmondásosnak tűnik, hogy egy rendkívül számításigényes gépi tanulási problémát egy viszonylag lassú, nyelvben történő megvalósítás követ, de a magyarázat igen logikus. A Python ezen a területen „ragasztónyelvként” működik. Értem ez alatt, hogy a nagy számítási igényű műveleteket, a mátrixszorzás, konvolúció vagy a gradiensszámítás, C, C++ és CUDA nyelven írt, optimalizált könyvtárak végzik a processzoron vagy a grafikus kártyán. A Python pedig ezen optimalizált könyvtárak kényelmes, magas szintű használtát biztosítja. Az interpreter által okozott műveletek többletideje elhanyagolható , köszönhetően ezen könyvtárak nativ implementációjának. A fejlesztő így egy könnyen írható nyelv kényelmét élvezi, a számítások mégis natív sebességgel futnak.

A neurális hálókat PyTorch [14] keretrendszerben valósítottam meg. Ez GPU-gyorsított tenzorműveleteket és automatikus differenciálást biztosít, a számítási gráfot pedig futás közben építi fel, ami megkönnyíti a fejlesztést és a hibakeresést. A választást az is indokolta, hogy a Stable-Baselines3 is PyTorch-ra épül, így a későbbi neurális hálók és az ágens hálói egy keretrendszerben, közös GPU-n futhatnak.

A Python mellett szólt végül de nem utolsó sorban, hogy a CARLA kliensoldali API-ja is elérhető Pythonban (2. fejezet) ámbár a kliens maga eredetileg C++ nyleven iródott. Így egyetlen nyelven valósulhatott meg a szimulátor vezérlése, a szenzoradatok feldolgozása, a neurális hálók tanítása és maga a megerősítéses tanulás is.

3.3. Soft Actor-Critic és a Stable-Baselines3

Kísérleteim alapjául a Soft Actor-Critic (SAC) algoritmust választottam. A választott algoritmus egy off-policy aktor–kritikus módszer, amely folytonos akciótérben sztochasztikus stratégiát tanul. Az eljárás minden állapothoz párosít egy valószínűségi eloszlást majd az akciók halmazából. Később ebből mintavételezi a végrehajtandó akciót.

A célja nemcsak a jutalom, hanem a stratégia entrópiájának maximalizálása is:

$$J(\pi) = \sum_t \mathrm{E}\Big[\, \gamma^t \Big(r(s_t, a_t) + \alpha\, \mathscr{H}\big(\pi(\cdot \mid s_t)\big)\Big)\Big], \tag{2}$$

ahol s_t a t-edik időlépés állapota, a_t az ekkor választott akció, $r(s_t, a_t)$ az érte kapott jutalom, $\pi(\cdot \mid s_t)$ a stratégia által adott akcióeloszlás, $\mathscr{H}$ ennek entrópiája (azaz mennyire szétterült az akcióeloszlás), α pedig a *hőmérséklet*, amely az entrópia súlyát adja meg a jutalomhoz képest. A γ diszkontfaktor (nálam 0,98) a későbbi jutalmakat kisebb súllyal veszi figyelembe. A várható érték a stratégia követésével bejárt állapotokra és akciókra vonatkozik.

A fentiekben elmondottak alapján az ágens addig marad változatos, amíg ez nem rontja érdemben a jutalmat. Így a felfedezés nagyrészt magából a célfüggvényből következik: az ágens kevésbé hajlamos korán leragadni egy rossz megoldásnál, és nem kötelezi el magát idő előtt egyetlen viselkedés mellett (például hogy most előzzön vagy kicsit később). Az SB3 megvalósításában a stratégia egy Gauss-eloszlás, amelynek mintáit a tanh függvény szorítja a megengedett akciótartományba. A felfedezést nálam ezen felül az állapotfüggő felfedezés (gSDE) [15] segíti: a zaj az aktuális állapottól függ, és nem minden lépésben sorsolódik újra, hanem csak 8 lépésenként, így a kormány- és gázmozdulatok simábbak, nem remegnek lépésről lépésre.

A tanítást két kritikus (Q-háló) segíti ezek alapértelmezetten implementálva vannak az STB3-ban. Ezek becslik a Q-függvényt, két kritikus egy közös jellemzőkinyerő hálón osztozik a szenzoradatokat feldolgozó csak a rá épül külön rétegként. A tanítás célértékét a két becslés kisebbikéből számolja, ami csökkenti a Q-értékek túlbecslését. A tanítás a visszajátszási pufferből vett kötegeken történik. Az α értékét az algoritmus automatikusan hangolja [16]: úgy állítja, hogy a stratégia entrópiája egy előre adott célérték közelében maradjon (az SB3-ban ez az akciótér dimenziójának mínusz egyszerese, nálam -2).

Az algoritmust nem magam implementáltam, hanem a Python Stable-Baselines3 (SB3) [17] könyvtárát használtam könnyenhasználhatósága valamint népszerűsége okán. Az SB3 egy megfelelően kitesztelt PyTorch-alapú megvalósítás emiatt a munkám a hatékony környezeti jellemzők kinyerésére és a megfelelően finomhangolt jutalomra összpontosulhatott. Az SB3 a Gymnasium [18] szabványú környezetekkel működik, amelyek a `reset` és `step` függvényeken keresztül indítanak epizódot és hajtanak végre akciót. A szabvány megkülönbözteti a valódi végállapotot (*terminated*) az olyan megszakítástól (*truncated*), amely után a vezetés folytatható

lenne; utóbbinál az SB3 a következő állapot becsült értékével számol tovább. Nálam valódi végállapot az ütközés, a pályaelhagyás, a megállás és a túl gyors haladás, megszakítás pedig az útvonal végének vagy a kitűzött távolságnak az elérése.

Megvalósításaim:

- a CARLA-környezet Gymnasium interfésszel (jármű, szenzorok, útvonalak, forgalom, epizódkezelés);
- a megfigyelések előállítása a szenzoradatokból (4. fejezet);
- a jutalomfüggvény és az epizódvég feltételei (5. fejezet);
- a kamera- és LiDAR-jellemzőket összegző, majd a járműadatokkal és az útvonal jellemzőivel összefűző háló;
- a fontosabb mutatókat TensorBoardba naplózó callbackek.

4. Szenzoradatok előfeldozása, Autóenkóderek

Az robotikai eszközök (például autók), környezetüket rájuk erősített szenzorok segítségével érzékelik. Ezen eszközök halamzának segítségével valósítják meg a gépilátás és érzékelés folyamatát. Míg az emberek saját érkezszerveikkel jutnak információhoz a külvilágról a robotikai eszközök, szenzor rendszereikkel végzik el ezt az igen kritikus lépést a tájékzódás során. A szenzoradatok a pontos érzékelés okánál fogva nagyon sokrétüek és sűrűek. Emiatt nagy számítási kapacitást igénnyel feldolgozásuk során valamint általában átesnek egy adattisztiási folyamaton is. Az említett folyamatok szükségessége abban nyilvánul hogy az ágens önmagában nem képes feldolgozni ezt a rengeteg információt, valamintha mégis ezek a feldolgozási folyamatok nagyba lassíthatják a döntés megszületését.

Ebben a fejezetben bemutatásra kerülnke az általam használt szenzorok, valamint az is hogy miként végeztem el az előfeldolgozási folyamatokat. Ágensem érzékelésért 2 fő szensor felelős, egy LiDAR valamint egy RGB kamera. Választásom azért esett ezekre az eszközökre mert ezek tökéletesen elegendőek arra, hogy megfelelőképp megfigyelje a forgalomat. A kamera a vizuális észlelés, hogy lesz előttem egy jármű, valamint azért felelős, hogy az ágens pontosan lássa a sávok felfestését az útburkolatom. A LiDAR szenzoradatok a térbeli letapogatásért felelős, információt szolgáltat például a megelőzendő jármű fizikai dimenzióiról. Akkor lehet hasznos hogy ne probálja túl korán befejezni az előzési manővert.

A CARLA mindkét szenzort valósághűen szimulálja [6]. A kamera képét az Unreal Engine grafikus motorja rendereli, és a valódi kamerákra jellemző hatásokat is utánozza: automatikus expozíciót, mozgási elmosódást, becsillanást, mélységélességet, stb. A LiDAR-t sugárkövetéssel szimulálja: a csatornák száma, a hatótávolság, a látószög és a forgási frekvencia a valódi

eszközökhöz hasonlóan beállítható, a mérésekhez zaj adható, és a pontok egy része véletlenszerűen elveszik, ahogy a valóságban is.

Előzetes tapasztalataim alapján nem optimális a nyers szenzoradatokat közvetlenül a modellnek adni. Egyetlen kamerakép is több tízezer számértékből ami igen megterheli a modellt futását a LiDAR pedig több tízezer pontból áll ráadásul a pontok száma és sorrendje lépésről lépésre változik. Ha az ágens ezeket nyersen kapná, a jutalomjelből kellene megtanulnia kinyerni a szenzorokból a lényeges információkat és vezetni. Ezek mellett a nagy bemenet a visszajátszási puffer memóriaigényét is jelentősen megnövelné.

Ezért saját, tömör beágyazásokat készítettem a szenzoradatokból, ehhez autoenkódereket alkalmaztam. Az autoenkóder egy kódolóból és egy dekódolóból álló háló, amely úgy tanul, hogy a bemenetét egy kis méretű belső reprezentáción (látens téren) keresztül próbálja visszaállítani. Az 1gy kinyert látens vektor vagy címke márcsak az adatok lényegi információit tartalmaza. A tanítás után a kódoló kimenete adja az ágensnek a bemenetét, így a tanulás elegendő ha a vezetés elsajátítására összpontosulhat.

Az autoenkóderek tanításához saját adatgyűjtő programot használtam és írtam. Ez a Town04 pályán, 200 forgalmi jármű között vezeti az autót, ugyanazokkal a szenzorbeállításokkal rögzít, mint amelyeket az ágens a tanulás során lát. A CARLA-ban elérhető autopilot, ami mindig pontosan a sáv közepén halad, az autót a sávon belül folyamatosan jobbra-balra térítettem, és időnként véletlen kormánymozdulatokat is adtam neki, így az adatban a sávból kisodródott, ferdén álló helyzetek is szerepelnek.

Ezzel biztosítva az adatok nagyobb varianciáját. A szimuláció 20 Hz-en fut, de csak minden negyedik lépést mentettem (5 Hz), erre szükség volt mivel az egymást közvetlenül követő képek szinte azonosak, és fölöslegesen növelnék az adat méretét. Összesen 40 211 egymáshoz tartozó kamera–LiDAR párt gyűjtöttem (kb. 15 GB). Az összetartozó kép és pontfelhő fájlneve a szimulációs képkocka sorszáma, így biztosan ugyanahhoz a pillanathoz tartoznak. Ezen az adathalmazon mindkét szenzorhoz három különböző autoenkódert tanítottam és hasonlítottam össze, hogy kiválasszam a legjobbat.

4.1. Kamera

Az ágens kamerája a CARLA szimulált RGB kamerája (`sensor.camera.rgb`), amelynek a helyzete a járműrvőn a szélvédő mögött van. Az autó testének középpontjához képest 1,6 m-rel előrébb és 1,7 m magasan, menetirányba néz.

A képet 160×80 pixeles felbontásban, 110°-os vízszintes látószöggel, a szimulációval szinkronban 20 Hz-en rögzíti (3. ábra). Ez a relatíve kisebb felbontás tökéletesen elegendő a sávjelek valamint a forgalom többi szereplőjének felismeréséhez.



3. ábra. Példák az ágens által látott 160×80 kameraképekből

Miután kész voltam a tanító adatok begyüjtésével kiválasztottam 3 különböző architektúrára épülő autoenkódert, hogy ezekből kiválasszam a számomra legjobbat. A három modellt az egyenlő feltételek mellett ugyanazon a rekonstrukciós feladaton tanítottam. A bemeneti és a visszaállított kép közötti minden esetben átlagos négyzetes hibát (MSE) mértem majd ezt minimalizáltam, Adam optimalizálóval.

A modellek bemenete a 160×80 pixeles RGB kép, a pixelértékei normalizáláson estek át így ezután a pixelek $[0, 1]$ tartományba estek. A 40 211 képet véletlenszerűen megkevertem és 70/15/15 arányban osztottam fel. Ezek után 28 149 kép került a tanító-, 6031 a validációs és 6031 a teszthalmazokba. A keverésre azért volt szükség, mert az egymást követő képek szinte azonosak, és enélkül a halmazok a pálya más-más szakaszait tartalmaznák.

Az eljárás során a validációs halmazon legjobbnak bizonyult modellt mentettem el. Az alábbi ábrák mindegyike ugyanazt a négy, a tanítás során félretett teszthalmazból származó képet mutatja, ez által a modellek rekonstrukciói közvetlenül összevethetők.

4.1.1. Konvolúciós autoenkóder

Első választottam egy egyszerű konvolúció autoenkóderre esett amely szerkezetileg a következő. A kódoló négy konvolúciós blokkból áll (4×4-es kernel, 2-es lépésköz, batch-normalizálás, LeakyReLU), amelyek mindegyike felezi a kép méretét, ezek által 3×80×160-as bemenetből 256×5×10-es jellemzőtérkép lesz. Ez egy teljesen összekötött réteg amely vetítésre kerül a látens vektorba a kinyert jellemzőkkel. A dekódoló ennek tükörképe transzponált konvolúciókkal, a kimenetet egy szigmoid függvény adja és szorítja a $[0, 1]$ es tartományba. Egyszerű, sekély felépítése miatt a három modell közül ez a leggyorsabb. A rekonstrukción (4. ábra) jól kivehetők a sávjelek, az útnak a széle és a járművek egyaránt jól kivehetők. A háttér apró részletei mosódnak

el de az jelen esetben amúgyis zajnak számít.



4. ábra. A konvolúciós autoenkóder rekonstrukciói a teszthalmazon

4.1.2. Reziduális autoenkóder

Második autoenkóder jelöltem egy Reziduális blokkokra épülő modell. Ebben a rétegek kimenetéhez hozzáadódik a bemenetük ($x + F(x)$); ez mélyebb hálóknál is stabil tanulást tesz lehetővé [19]. A háló négy felbontási szinten dolgozik, szintenként két reziduális blokkal, és többfelbontású átkötéseket is felhasznál. Minden szint kimenete közvetlenül a szűk keresztmetszet elé is eljut, hogy ez által a finom részletek is bekerülhessenek a látens reprezentációba. A három modell közül ennek van a legkevesebb paramétere (2,4 millió). A rekonstrukció (5. ábra) erősen elmosott: az út nagyjából felismerhető, de a jármű körvonala szétfolyik, a környezet pedig általános, ködös foltokká válik.



5. ábra. A reziduális autoenkóder rekonstrukciói a teszthalmazon

4.1.3. VGG-alapú autoenkóder

A harmadik megvalósítás egy a VGG-19 hálózat [20] első négy blokkja (a `relu4_1` rétegig), az ImageNet adathalmazon előtanított súlyokkal. Előtanítottsága okán az alapvető képi jellemzők (élek, textúrák) felismerését a hálónak nem kell nulláról megtanulnia. A dekódoló a kódoló tükörképe. Mivel ez egy mély, normalizálás nélküli háló, kisebb tanulási rátával (10^{-4}) kellett tanítani. Éredkesség, hogy az előtanított súlyok nélkül egyáltalán nem tanult. A rekonstrukción (6. ábra) a sávjelek és a jármű megmaradnak, de elmosódnak, és a modell helyenként a jelenetre

jellemző, de valójában ott nem lévő elemeket rajzol. a második képen az épület és a téglafal helyén fákat rajzol amik valójában ott sincsenek.



6. ábra. A VGG-alapú autoenkóder rekonstrukciói a teszthalmazon

4.1.4. Összehasonlítás és a választott modell

A modelleket a 40 211 képből félretett, 6031 képes teszthalmazon hasonlítottam össze. Az MSE mellett a csúcsjel-zaj viszonyt (PSNR) is megfigyeltem amely a $[0,1]$ tartományba skálázott képekre a következő:

$$\mathrm{PSNR} = 10\log_{10}\frac{1}{\mathrm{MSE}}\ \mathrm{dB}, \tag{3}$$

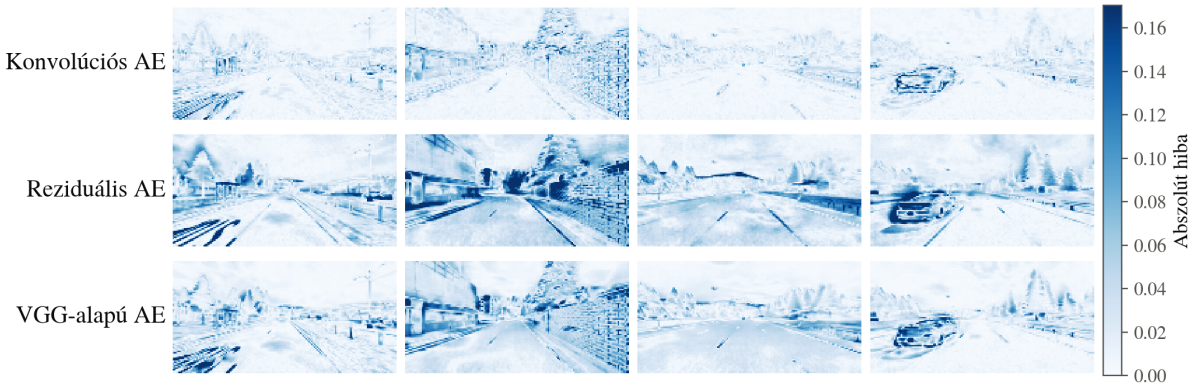
Ezen mérőszám minál nagyobb, annál pontosabb a visszaállított kép. A kódoló az ágens tanítása közben minden lépésben, ugyanazon a GPU-n fut így a kódolási idő is szignifikáns jellemzők közés soroltam. (1. táblázat).

1. táblázat. A három kamera-autoenkóder összehasonlítása a teszthalmazon

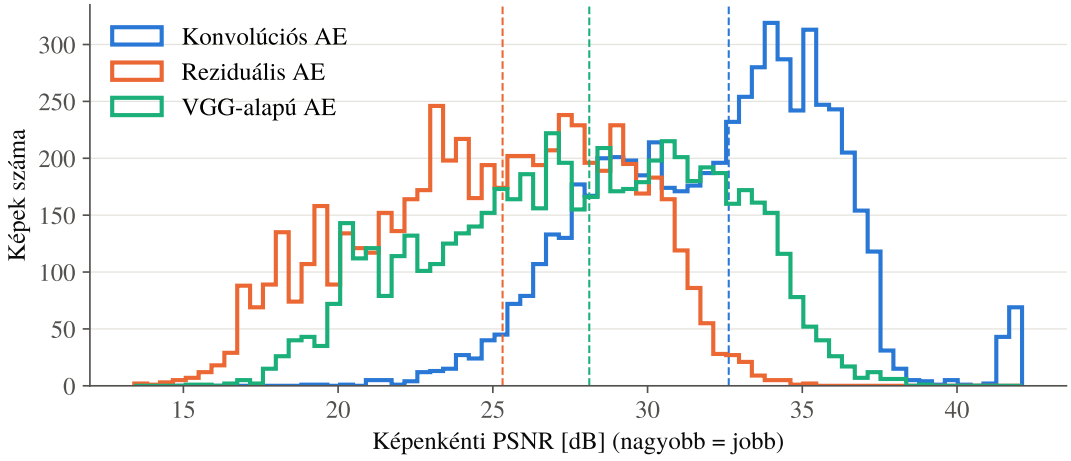
Modell	MSE	PSNR [dB]	Paraméter [M]	Kódolás [ms]
Konvolúciós AE	**0,00088**	**30,58**	4,5	**0,27**
Reziduális AE	0,00495	23,06	**2,4**	0,93
VGG-alapú AE	0,00278	25,56	8,7	1,01

A fenti táblázat alapján a konvolúciós autoenkóder hibája a legkisebb. Az MSE-je harmada a VGG-alapú megoldáshoz képest és kevesebb, mint ötöde a reziduális modellének, ami 5, illetve 7,5 dB-lel jobb PSNR-t jelent. A különbség nem csak átlagban áll fenn, a képenkénti párosítás során kiderült, hogy a konvolúciós megoldás a teszthalmaz képeinek 97,4%-án kisebb a hibája, mint a VGG-alapú modellé, a reziduális modellnél pedig minden képen jobb (8. ábra).

A hibatérképeken (7. ábra) az is látszik, hogy a konvolúciós modell rekonstrukciója a legkevésbé zajos. A hiba nála csak a tárgyak éleinél jelenik meg vékony vonalakként. A többi modellnél esetében nagyobb, összefüggő foltokban, például a jármű és a sávjelek környékén, vagyis ott, ahol az ágensnek a legpontosabb információra volna szüksége.



7. ábra. A rekonstrukciók abszolút hibája képpontonként, közös skálán (sötétebb = nagyobb hiba)



8. ábra. A képenkénti PSNR eloszlása a teszthalmazon; a szaggatott vonalak a modellek mediánját jelölik

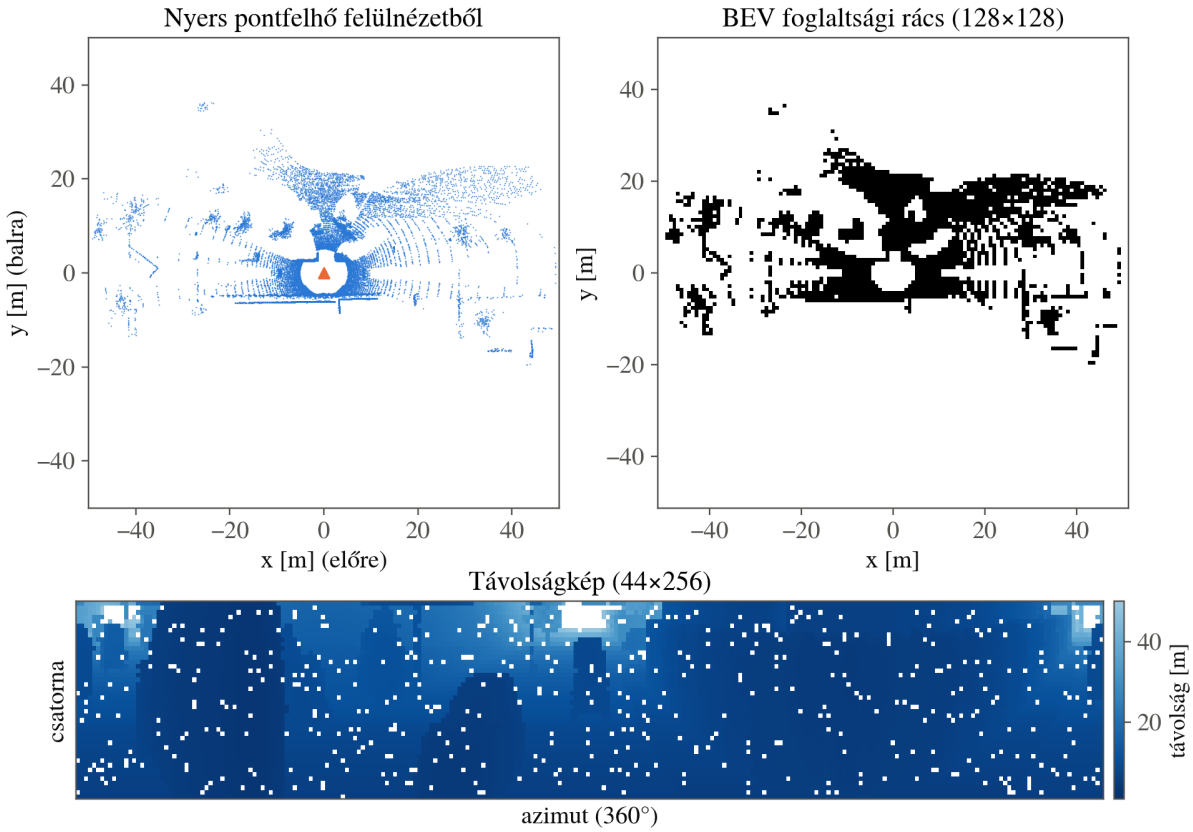
Ezen okoknál fogva választásom a konvolúciós autoenkóderre esett. A legjobb rekonstrukció mellett ez a leggyorsabb is. Egy kép kódolása kb. 0,27 ms, a másik két modellnél ez közel négyszer annyi.

4.2. LiDAR

Amint a fentiekben is említettem az általam választott második gépi érzékelési eljárás a térbeli letapogatás a LiDAR szenzor által. Az ágens LiDAR-ja a CARLA-ban alapból szimulált sugárkövetéses LiDAR szenzor. (`sensor.lidar.ray_cast`). Helyzetét tekintva jármű tetején,

középpontja felett 2,4 m magasan helyezkedik el. A szenzor 64 csatornás, függőlegesen +10° és −25° között, vízszintesen 360°-ban, 50 m-es hatótávolságig mér. Forgási frekvenciáját nézve a szimulációval azonos 20 Hz-re állítottam, így minden szimulációs lépésben pontosan egy teljes körülfordulásra elegendő adata érkezik. Egy fordulat 64×1024 sugárból áll, ezek egy részét a CARLA a valós szenzorokhoz hasonlóan véletlenszerűen elejti, így képkockánként mintegy 30 ezer pont marad.

A 3D pontfelhő általánosságban rendezetlen pont halmazként érkezik a szenzorból. A pontok száma jelenetről jelenetre változik, ez a változatosság egy neurális háló számára közvetlenül nem feldolgozható. A kameránál látottakhoz hasonlóan itt is az adat látens térbeli beágyazása volt a tervem. Szintén választottam három különböző architektúrájú autoenkódert majd ezeket mértem össze, a lényegi különbség, hogy mindegyik más formában kapja meg a jelenetet (9. ábra). felülnézeti rácsként, távolságképként, illetve közvetlenül pontfelhőként. Mindhárom modellt azonos adathalmazon, 40 epochon át tanítottam, és a validációs halmazon legjobbnak bizonyult modellt mentettem el. Az adatokat itt is véletlenszerű keverés után 70/15/15 arányban osztottam tanító-, validációs és teszthalmazra. A hibafüggvény modellenként eltér, mert mindegyik más formában állítja vissza az adatokat

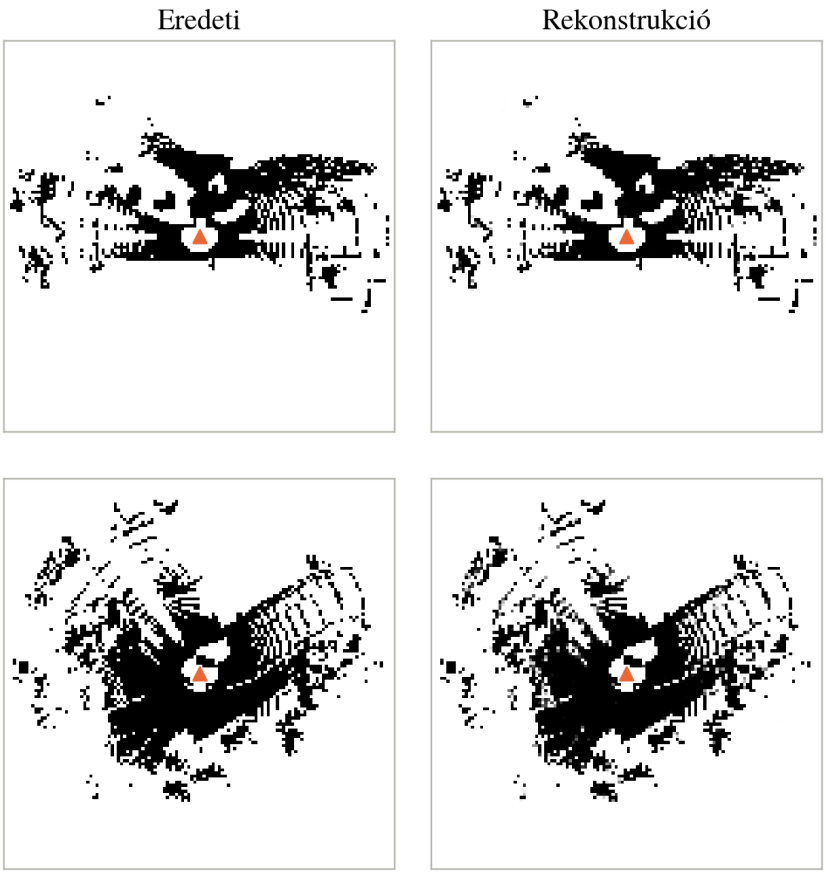


9. ábra. Ugyanaz a LiDAR-képkocka a három modell bemeneti formájában; a háromszög a saját járművet jelöli, a távolságképen a fehér cellákból nem érkezett visszaverődés

4.2.1. BEV-alapú autoenkóder

Első megoldásként egy relatíve népszerű megoldást választottam, mégpedig a pontfelhőt felülnézeti (BEV) rácsra valóvetítését. A jármű körüli 102,4×102,4 m-es területet 0,8 m-es cellákra osztottam (128×128 cella), majd cellánként három értéket számoltam.Elsőként foglalt-e, majd megnéztem benne lévő legmagasabb és legalacsonyabb pont magasságát. Erre a már 2 dimenziós képre egy konvolúciós autoenkódert építettem, amely a képet egy jóval kisebb jellemzőtérképbe tömörítette. Mivel a foglaltság 0/1 érték, erre a hibafüggvény bináris keresztentrópia.

A rekonstrukció (10. ábra) szinte hibátlan, a foglalt cellák 98–99%-át helyesen adja vissza. Sajnos ennek a megoldásnak több hátránya is, hogy a rács felbontása rögzített, a 0,8 m-nél kisebb részletek elvesznek valamint a függőleges 3D szerkezetek nagy része már a bemenetből hiányzik. A kódoló kimenete, vagyis a látens reprezentáció egy 256×8×8-as jellemzőtérkép, ami összesen 16 384 érték.



10. ábra. A BEV-alapú autoenkóder rekonstrukciói (foglalt cella: fekete)

4.2.2. Gráf-alapú autoenkóder

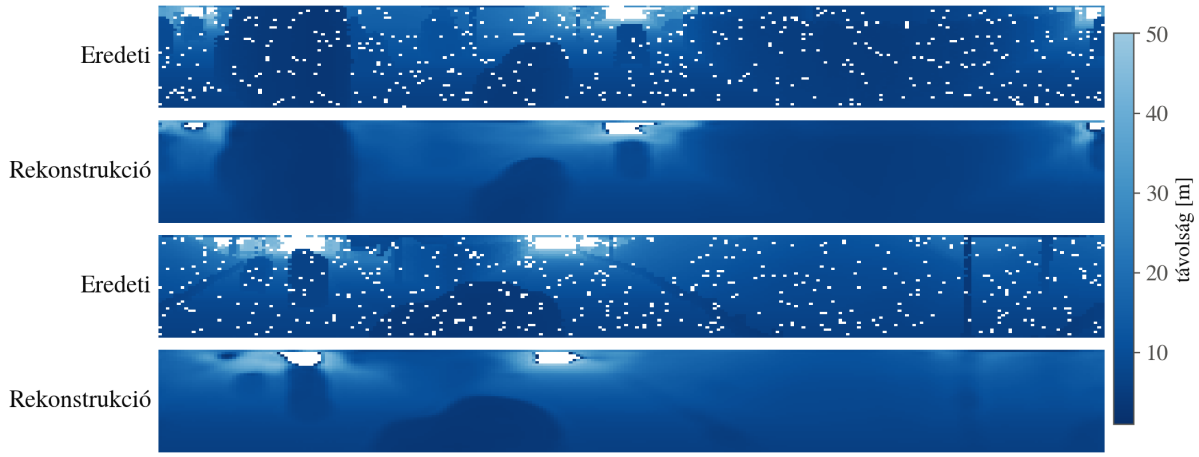
A második megoldás során a pontfelhőt távolságképpé alakítom. Ez annyit tesz, hogy minden sugárirány egy cella, az oszlop lesz a hozzátartozó irányszög, a sor pedig az emelkedési szög. A cellák értékét pedig a mért távolság adja meg. A CARLA LiDAR-ja szabályos sugárrácsban mér, ez által maga a vetítés kevésbe veszteséges mint a BEV esetén. Az ég felé tekintő felső sorok nem rendelkeznek számomra fontos információval ezért levágtam. Az így keletkezett kép 44×256 cellás, a távolságokat pedig logaritmikus skálán a $[-1,1]$ tartományba normáltam.

A kódoló ötletét a TopoLiDM [21] LiDAR-modellből vettem. A hálózat a konvolúciós rétegek mellett gráfrétegeket is tartalmaz, ezek a rétegek minden jellemzőt a hozzá leginkább hasonló szomszédjaival vetnek össze, így a térben összetartozó részeket akkor is együtt kezelik, ha a képen távol esnek egymástól. A kép két szélét körkörös konvolúció köti össze, hiszem a 360°-os körben való szomszédság fontos részlet.

A kódoló kimenete $16\times11\times32$-es jellemzőtérkép. Hibafüggvénynek az L1-et választottam amely átlagos abszolút hibát számolja:

$$L_1 = \frac{1}{N}\sum_{i=1}^{N} |x_i - \hat{x}_i|\,, \tag{4}$$

A képletben x_i az eredeti, $\hat{x}_i$ a visszaállított távolság az i-edik cellában. Az N pedig a cellák száma, az eljárás minden cella eltérését az eltérés nagyságával arányosan, egyenlő súllyal veszi figyelembe, így egy-egy nagyobb hiba nem nyomja el a többit, valamint a ritka, kiugró mérési hibák sem befolyásolhatják a tanulást. A hiba akkor a legkisebb, amikor a modell cellánként a legvalószínűbb értéket (a mediánt) becsli. Ez a távolságképen különösen fontos, mert egy tárgy széle éles ugrásként jelenik meg. Az üres, visszaverődés nélküli cellák ugyanakkora súllyal számítanak, így a modell azt is megtanulja, hol végződik egy tárgy. A rekonstrukción (11. ábra) a közeli járművek és az út széle jól kivehetők, a finom részletek viszont láthatóan kevésbé maradnak hangsúlyosak.



11. ábra. A gráf-alapú autoenkóder rekonstrukciói két képkockán (sötétebb = közelebb, fehér = nincs visszaverődés)

4.2.3. Point-MAE

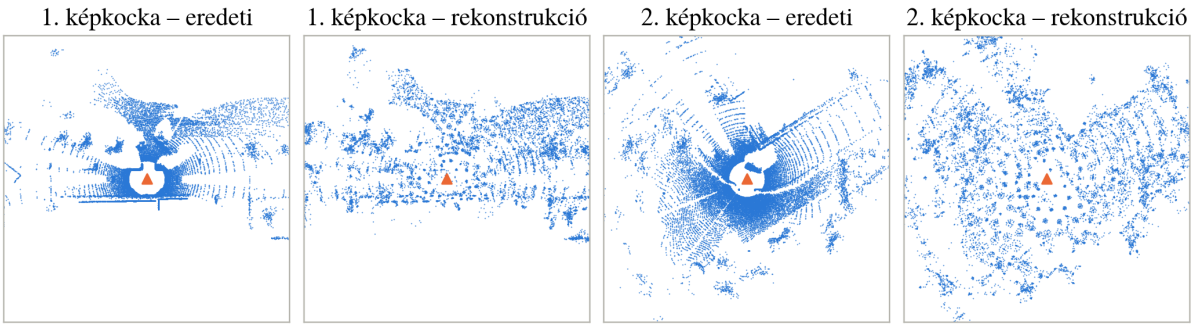
A harmadik választottam a Point-MAE [22] modell ami képeknél már sikeresen bevált maszkolt autoenkóder ötletét viszi át pontfelhőkre, és közvetlenül a pontfelhőn dolgozik, képkockánként 24 576 véletlenszerűen kiválasztott ponttal.

A pontfelhőt 512 darab, egyenként 32 pontos foltra bontja. Ezek után a foltok középpontjait legtávolabbi-pont mintavételezéssel választja ki. Erre azért van okvetlenül szükség, hogy egyenletesen lefedjék a jelenetet. Minden ponthoz a 32 legközelebbi szomszédot rendeli. Minden foltot egy kis háló egy 192 elemű vektorrá (tokenné) alakít, a foltok helyét pedig külön pozíciókódolás segítségével adja meg.

Ezen tokeneket egy Transformer kódoló dolgozza fel. Az eljárás során a foltok 60%-át elrejti, és a modellnek a látható foltokból kell visszaállítania a rejtetteket. Ezen modell esetén a hibát Chamfer távolsággal mértem, mely mindkét pontfelhő minden pontjához megkeresi a másik felhő legközelebbi pontját. Ezeknek a pontoknak a távolságát átlagolja, ez által nem függ a pontok sorrendjétől. A kódoló kimenete, vagyis a látens reprezentáció 512 token, egyenként 192 dimenzióval, ami összesen 98 304 érték.

Ez jelentősen több, magánál a bemenetnél is (24 576 pont $\times$ 3 koordináta = 73 728 érték), vagyis ez a modell nem tömörít. A maszkolt autoenkóder nem szűk keresztmetszettel, hanem a kitakart foltok visszajóslásával tanult így mint kiderült számomra használhatatlan lett. A legtávolabbi-pont mintavételezés ráadásul véletlen pontból indul, ezért ugyanaz a képkocka kétszer kódolva is más sorrendű, részben más foltokból álló tokensort ad.

A rekonstrukción(12. ábra) a jelenet nagy vonalai felismerhetők, de a pontok csomókba tömörülnek, és a járművek körvonala elvész.



12. ábra. A Point-MAE rekonstrukciói felülnézetből (a háromszög a saját jármű)

4.2.4. Összehasonlítás és a választott modell

A megfelelő LiDAR autoenkóder keresése elején 3 modellt készítettem, de a Point-MAE-t kihagytam az összehasonlításból, mert mint kiderült nem lett megfelelő ahoz amire én használnám. A látense (98 304 érték) nagyobb, mint maga a bemenet, vagyis nem tömörít.

Így a következőkben a másik két modellre fokuszálnék. Mindkét megoldás más-más bemenetet állít vissza, ezért a rekonstrukciós hibájuk nem vethető össze.

Ezen okból másképp fogtam meg azon kérdést, hogy vajon hogyan is válasszam ki a legjobb rekonstrukciót. Az alapegondolás onnan indult, hogy az ágens ugye nem a rekonstrukciót kapja meg, hanem a kódoló kimenetét, a látens minőségének elemzésére fektettem a hangsúlyt. Legfőbb célom volt kideríteni, hogy melyik eljárás képes a legjobban megőrizni a környezet geometriáját. Összesen 3db mérést végeztem az adathalmazból véletlenszerűen kiválasztott 2000db képkockákon.

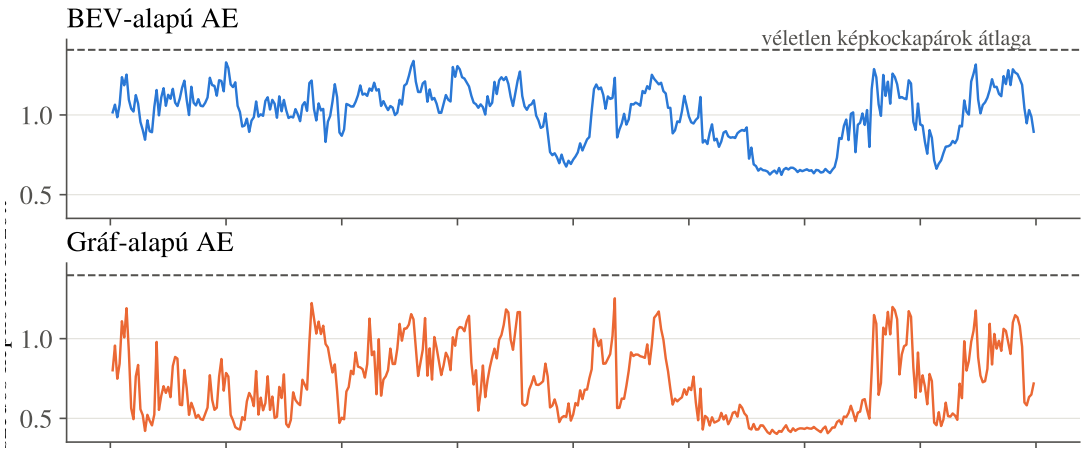
Az eljárások a következők:

- **Effektív rang** (RankMe [23]): a látens dimenziók mekkora része hordoz, gyakorlatilag azt mérem mennyire zajos a látens vektor (1 = teljes kihasználtság).
- **k-NN konzisztencia** [24]: a látens térben legközelebbi képkockák valóban hasonló jelenetet mutatnak-e; a szomszédok pontfelhői hányszor közelebb vannak egymáshoz, mint két véletlen képkockáé.
- **Időbeli simaság**: két egymást követő képkocka látense hányszor közelebb van egymáshoz, mint két véletlen képkockáé. Ha alig, a látens apró változásra is nagyot ugrik, ami az ágens számára zajos bemenet.

A gráf-alapú modell látense a legsimább (13. ábra): két egymást követő képkocka között nála a legkisebb a változás. Az effektív rangban a BEV-alapú modell a legjobb, de mindkét modell 0,5 felett van, tehát egyik látense sem omlott össze néhány dimenzióra.

2. táblázat. A két LiDAR-autoenkóder látens reprezentációjának összehasonlítása

Modell	RankMe	k-NN	Simaság
BEV-alapú AE	**0,725**	**1,51**×	1,44×
Gráf-alapú AE	0,504	**1,51**×	**1,86**×



13. ábra. A látens változása két szomszédos képkocka között, 400 egymást követő képkockán (5 Hz); a szaggatott vonal két véletlen képkocka látensének átlagos távolsága

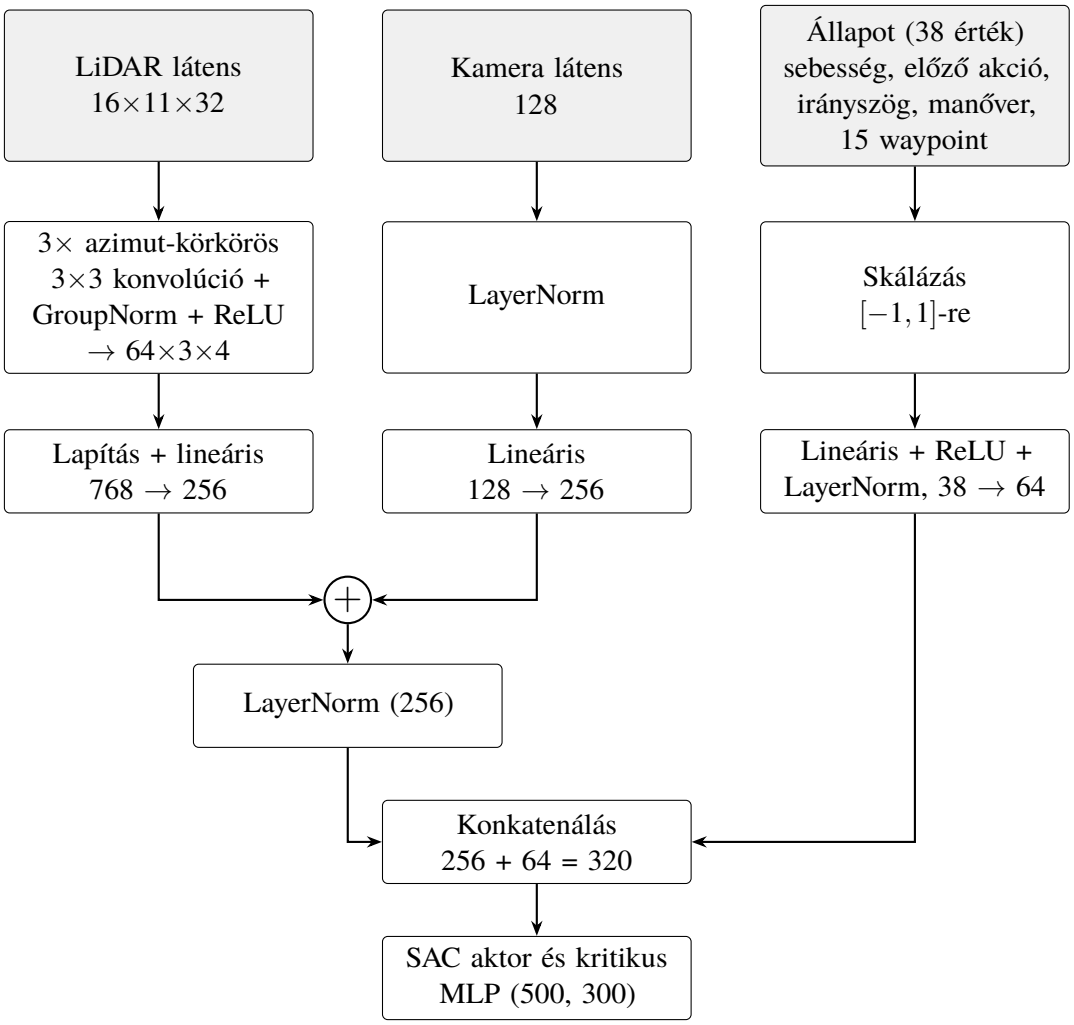
A fenti mérésekből következtetve, a gráf-alapú autoenkódert választottam. Ennek a látense követi a legpontosabban a jármű folyamatos mozgását. Ez a tulajdonság kiemelkedően fotnos az ágens számára mivel az egymást követő lépésekben következetes bemenetet kap.

4.3. Szenzorfuzio

A két autoenkóder látense jóval nagyobb, számokban is és nagyságrendileg is, mint az ágens többi bemenete. A LiDAR látens 16×11×32, azaz 5 632 érték, a kamera látense 128 érték, míg a jármű állapotát leíró megfigyelések összesen 38 számszerű értéket tartalmaznak.

Ha ezeket a Stable-Baselines3 beépített eljárására bíztam, hogy oldja meg a tanulást a látensek nagyságrendileg annyira elnyomták a többi jellet hogy nem mentem semmire a tanítás során, gyakorlatilag az ágens nem látta mire figyeljen.

Ezen problémától motiválva saját jellemzőkinyerőt (*feature extractor*) készítettem, amely három külön bemeneti ágon dolgozza fel a szenzorokból érkező adatokat, majd ezekt a végén ugy rendezi ahogy nekem tetszik. (14. ábra).



14. ábra. A saját jellemzőkinyerő felépítése; a szürke dobozok a bemenetek

LiDAR ág. A LiDAR szenzor dekódolójából származó látens nem egy sima vektor, hanem egy több dimenziós jellemzőtérkép. Ezt az ágens saját konvolúciós rétegei dolgozzák fel, amelyek az ő tanulása folyamán finomhangolodnak azon jellemzőkre amire tényleg szűkségesek a modellnek gyakorlatilag együtt tanul a modellel. Három 3×3-as konvolúciós réteg csökkenti a térkép méretét 16×11×32-ről 64×3×4-re, mindegyik után GroupNorm normalizálással és ReLU aktivációval. A kapott 768 értéket egy lineáris réteg 256 dimenzióra vetíti.

Kamera ág. A kamera látense már vektor, ezért itt elég egyetlen lineáris réteg, de a fentiekben is említettem, hogy a LiDAR jellemzők 256 dimenziósra csökkentek így ha össze akarom adni a 128 hosszú látensel akkor 256 dimenziósra fel kell skáláznom. Ezt szintén egy Lineáris réteggel oldtam meg.

Fúzió. A lényeges rész a fuzios eljárás maga esetemben a két szenzorágat összeadással vezettem össze. Azért ezt a megoldást preferáltam az összefűzéssel ellentétben, mert a jellemzőkinyerő végén a kimenet alapértelmezetten konkatenálással áll össze, fölösleges még egy összefüzést itt is alkalmazni. Mivel a két ág azonos, 256 hosszú vektort ad, az összeadáshoz nincs szükség további vetítő rétegre, és a valamint az egyesített szenzor jellemző vektor mérete sem nő. Az összeg után egy LayerNorm rögzíti a vektor teljes skáláját, mert a gráf-alapú kódoló kimenete sem korlátos. Viszont a két szenzor ág egymáshoz viszonyított súlyát nem állítom manuálisan, ezt az ágensre bízom honnan tud több információra szert tenni.

Állapot ág. A maradék további megfigyelés került utoljára A jármű sebessége, az előző lépés kormány- és gázértéke, a következő waypoint irányszöge, a manőver one-hot kódolva, valamint a következő 15 waypoint relatív koordinátái.

Ezeket a jellemzőket a megfigyelési tér határai alapján a $[-1, 1]$ tartományba skáláztam hogy a nagy értékű jelek ne nyomják el a kicsiket, majd egy lineáris réteg, ReLU és LayerNorm 64 dimenzióra képezi le őket.

A jellemzőkinyerő kimenete a 256 hosszú szenzorvektor és a 64 hosszú állapotvektor összefűzése, összesen 320 érték. Ez kerül a SAC aktor- és kritikushálózatának bemenetére, amelyek két, 500 és 300 neuronos rejtett rétegből állnak. Így a szenzorinformáció a bemenet nagyobb részét adja, de az állapotjelek is külön, saját ágon jutnak el a döntéshozó hálózatig.

5. Jutalom függvény

A feladat komplexitása a munka ezen pontján igencsak megnőttt, mivel a jutalom függvény segítségével kell megmagyaráznom az ágensnek az előzés műveletét. Értem ez alatt, hogy az ágensnek biztonságosan, sávot kell tartania, valamint nála lassabb járműveket szabályosan, balról kell megelőznie, természetesen ha erre van lehetősége. A feladat megfelelő körbejárására és felderítésére, 5db különböző jutalom függvényt készítettem amelyek a későbbiekben a különböző modelleket is jelölik. Az (R1–R3) függvények részletesebben minden forgalmi helyzetre külön, kézzel általam formált tagot tartalmaznak ahol midne tagnak saját célja van. Az utolsó két esertben egy másféle megközelítést alkalmaztam (R4–R5) amely elhagyja a kézi tagokat, egyetlen fizikai mennyiséget jutalmaz.

Ugyanakkor a különböző ágensek tanítása során voltak közös konstans metrikák mint például, az epizódzáró büntetés. A járművel való ütközés esetén -30, a többi esetben (az útról való letérés, megállás, a sebességhatár túllépése) -10. Minden modell esetén közös a kormányrángatásért kiszabott büntetés, amelyet a kormányjel lépésenkénti változásának kétszereséből számolok. Az előzési manővert felismerő állapotgép, ahol csak következő van engedélyezve a balra, szaggatott vonalon áthaladás ha szabad a sáv kihúzódhat és megkísérelheti az előzési manővert.

5.1. R1: sávtartás és szabályos előzés

Az alapjutalom amit kap szabad úton négy, nullától egyig terjedő tényező szorzata:

$$r = v^{1,9} \cdot c \cdot a \cdot s, \tag{5}$$

ahol v a sebesség (50 km/h-nál 1, alatta lineáris), c a sávközép-tényező (a sáv közepén 1, 2 m-re tőle 0), a az irányszög-tényező (az útvonal irányába nézve 1), s pedig a simasági tényező, amely azért felelős hogy a modell ne végezzen hírtelen fölösleges kormány mozdulatokat amikor a sávjában közlekedik. A tagokat szorzás utolján fuzionálom ennek oka, hogy bármelyik kiesése nullázza a jutalmat. Az ágnest így kényszerítem arra, hogy egyszerre kell mindegyiket teljesítenie. Az 1.9 es kivehető oka a következő, az 1-nél nagyobb kitevő hatása az, hogy a lassú haladás aránytalanul keveset ér: 25 km/h sebességnél $v = 0{,}5$, a sebességtag értéke viszont már csak $0{,}5^{1,9} \approx 0{,}27$, nem pedig a fele. Ez erősebben ösztönöz a sebesség kihasználására, mint egy lineáris tag tenné, ugyanakkor az előzéskor is hatékonyabb mert vonzobbá teszi mert végsősoron a cél, hogy az ágens elkerülje a lassan forgalmirésztvevőket.

A következő eset, amikor az ágens előtt van egy forgalmi szereplő de nem engedélyezett az előzés. A saját sávjában 40 m-en belül lassabb jármű megy előtte(foglalt vagy záróvonallal elválasztott szomszédos sáv), akkor követnie kell nem sazbad baleset történjen. Ilyenkor a sebesség helyett helyett a biztonságos követési távolság megtalálását jutalamzom, hogy az ágens megfelelően kövesse aki előtte van.

$$r_{\text{követés}} = 0{,}8\, g(h)\, m \cdot c \cdot a \cdot s + p_{\text{TTC}}, \tag{6}$$

ahol h a követési időköz (a távolság és a saját sebesség hányadosa az előtte haladóhoz képest), g pedig egy lognormális alakú függvény, amelynek maximuma 2 s-nál van. A 0,8-as szorzót azért alkalmazom, hogy a tökéletes követés is kevesebbet érjen a szabad haladásnáL. A sáv közepén, 2 s-os időközzel, az előtte haladóval azonos sebességgel követve a lépésjutalom 0,8, míg szabad úton 50 km/h-val 1,0. Így az akkor is megéri előzni ha egyébként hibátlanul követ, ezzel kiküszöbölve, hogy szabad ó út esetén merjen előzni.

Az m az ágens és az előtte levő jármű sebesség hányadosa ami fentről 1re van korlátozva:

$$m = \min\left(\frac{v_{\text{ego}} + 1}{v_{\text{lead}} + 1},\ 1\right). \tag{7}$$

alapból a g egymagában kijátszható. A h követési időköz a távolság és a *saját* sebesség hányadosa, ezért lassítással is növelhető: ha az ágens lefékezne és lemaradna, h 2 s fölé nőne, a jutalma mégsem csökkenne. Az m ezt hivatott javítani a lassabb haladás arányosan bünteti, az előtte haladó sebességét elérve, hogy kövesse amíg kell utána már ne érje meg követni. A két tényező együtt azt várja el, hogy az ügynök tartsa a kétmásodperces követési időt, és közben

haladjon ugyanolyan gyorsan, mint az előtte lévő jármű. A számlálóban és a nevezőben szereplő +1es eltolás technikai okból kell, nélküle álló jármű esetén nullával kellene osztani.

A kétmásodperces optimum és a g lognormális alakját a szakirodalomból vettem. A h időköz csak pozitív lehet, eloszlása pedig jobbra ferde,mivel ayz autóvezetők nagyrészt inkább közelebb haladnak az előttük haladóhhoz mint lemaradjanak. Az empirikus forgalmi mérések kimutatják ezt amit a lognormális eloszlás írj le a legjobban [25], a g-ben szereplő $\sigma = 0{,}4365$ szórás pedig az NGSIM adathalmazon, 1341 valós követési eseményre illesztett eloszlás becsült paramétere [26]. A jutalom így a tényleges emberi vezetői viselkedéshez igazodik.

A p_{TTC} (Time to collision) büntetés azt a helyzetet kezeli, amit a g nem lát. A pillanatnyi követési időköz lehet ideális úgy is, hogy az ego jármű közben gyorsan közeledik az előtte haladóhoz, tehát ütközés veszélye áll fenn. A büntetés az ütközésig hátralévő idő alapján jár, 7 s alatt lép életbe $0{,}3 \cdot \ln(t/7)$ alakban, és 1,5 s-nál éri el a $-0{,}46$-os maximumát. Ha az ügynök nem közeledik az előtte haladóhoz, ez az idő végtelen, és büntetés sem jár.

Előzési manőver végrehajtása közben adott lépésenkénti jutalom

$$r_{\text{előzés}} = (0{,}3 + 0{,}6\, v^{1{,}9}) \cdot l \cdot k, \tag{8}$$

ahol l a célsáv közepétől mért távolságot bünteti, k pedig az elhúzódó manővert: 15 s után, illetve az elhaladást követően sávonként 3 s után négy másodperc alatt 1-ről 0,3-ra esik. A 0,3-as alaptag biztosítja, hogy a lassú előzés is jobban megérje, mint a lassabb jármű mögötti kullogás. A szabályosan befejezett előzésért járművenként egyszer $+5$ bónusz jár.

Külön tag bünteti azt, ha az ágens előzhetne, de nem teszi. Mely szerint ha 25 m-en belül halad az előtte lévő mögött, nem gyorsabb nála legalább 5 km/h-val, lépésenként $-0{,}10$ és $-0{,}18$ közötti büntetést kap, amely az idő múlásával nő. Ez az egyetlen eset, amikor a lépésjutalom nem szorzódik a sávtartási tényezőkkel: ellenkező esetben a rossz sávtartás csökkentené a büntetés mértékét.

5.2. R2: ütközésidő-büntetés előzhető helyzetben

Az R1-ben a p_{TTC} büntetés csak akkor járt, ha az előzés nem volt lehetséges. Előzhető helyzetben hiányzott, ezért megérte teljes sebességgel ráfutni az előttünk haladó járműre, és csak az utolsó pillanatban kihúzódni. Az R2 egyetlen változtatása, hogy ugyanezt a büntetést (7 s küszöb, 0,3 súly) az előzhető állapotra is kiterjeszti. A büntetés a kihúzódás első méterén fokozatosan szűnik meg, így a már megkezdett manővert nem akadályozza.

5.3. R3: hangolás bátrabb előzésre

Az R2-vel tanított modell nem mert határozottan előzni. A 3. táblázat felső része megmutatja, miért: a lassabb jármű mögött 25–40 m-rel lemaradva haladni büntetlen, sőt pozitív jutalmat

adott, miközben az előzés a legjobb esetben is csak 0,90-et ért, kevesebbet, mint a saját sávban elérhető 1,00. Az R3 ezen négy ponton változtat:

1. Az előzhető helyzetben járó ütközésidő-büntetés küszöbe 7 s-ról 3 s-ra csökken, súlya viszont 0,3-ról 0,6-ra nő. Így csak a tényleges késlekedést bünteti, a gyors ráfutást nem.

2. Az előzés lépésjutalma $(0{,}3 + 0{,}6\,v^{1{,}9})$ helyett $(0{,}4 + 0{,}7\,v^{1{,}9})$, vagyis 50 km/h-nál 0,90 helyett 1,10. Ezzel a kihúzódás értékesebb lesz, mint a saját sávban maradás.

3. A lemaradás büntetése már nem csak 25 m-en belül jár, hanem a teljes 40 m-es érzékelési tartományban.

4. A $+5$ bónusz két részre bomlik: $+2$ az elhaladáskor és $+3$ a szabályos visszasoroláskor. A 0,98-as diszkonttényező mellett az eredeti, csak a manőver végén érkező bónusz a kihúzódás pillanatában mindössze 0,66-ot ért; a kettébontás ezt körülbelül 1,13-ra növeli.

3. táblázat. Lépésjutalmak egy 25 km/h sebességgel haladó jármű mögött, szabad bal sáv mellett. Az értékek a sáv közepén, egyenes haladásnál, kormánybüntetés nélkül érvényesek.

Helyzet	R2	R3
Kullogás 25 km/h-val, 25 m-en belül	$-0{,}12$	$-0{,}12$
Lemaradás 25–40 m-rel, 25 km/h-val	$+0{,}27$	$-0{,}12$
Ráfutás 50 km/h-val, 30 m-re	0,86	1,00
Ráfutás 50 km/h-val, 15 m-re	0,65	0,80
Előzés 50 km/h-val a bal sáv közepén	0,90	1,10
Előzés 25 km/h-val a bal sáv közepén	0,46	0,59

5.4. R4: haladásalapú jutalom

Az R1–R3 változatok minden forgalmi helyzetre külön, kézzel hangolt tagot tartalmaznak. Az R4 ezek helyett egyetlen fizikai mennyiséget jutalmaz: az útvonal mentén megtett elmozdulást. A lépésjutalom minden megengedett sávban

$$r = v \cdot \cos\vartheta \cdot l \cdot k, \tag{9}$$

ahol v a sebességtényező, ϑ az útvonal irányától való eltérés szöge, l és k pedig a (8) egyenletből ismert sávtartási és elhúzódás-tényező. A koszinuszos tag a ferde haladást veszi figyelembe, visszafelé haladva negatív. A sebességben lineáris, tehát 25 km/h-nál feleannyi jutalom jár, mint 50-nél.

A feltételezés az, hogy a lassabb jármű mögött a haladás magától kisebb, ezért az előzés kézi formálás nélkül is megéri. Kimarad a követési távolság jutalmazása, az ütközésidő-büntetés, a

lemaradás büntetése, a sávközép- és a simasági szorzó, valamint az 1,9-es kitevő. Megmarad viszont minden, ami a feladat szabálya és nem formálás: az epizódzáró büntetések, az előzési állapotgép, a rossz sávért járó $-0{,}5$, a kormányrángatás büntetése és a szabályos előzésért járó $+5$ bónusz. Az ütközést így kizárólag a -30 terminális büntetés tanítja.

5.5. R5: potenciálalapú jutalomformálás

Az R5 az R4 kiegészítése Ng, Harada és Russell potenciálalapú formálásával [27]. A módszer szerint a

$$F(s,s') = \gamma\Phi(s') - \Phi(s) \tag{10}$$

alakú tag, ahol Φ csak az állapottól függ, bizonyítottan nem változtatja meg, melyik stratégia optimális: a diszkontált összegben kiejti magát, csak előbbre hozza a jutalmat. Az R3 bónuszkettébontása ugyanezt próbálta kézzel elérni, itt elméletileg megalapozott formában történik.

A potenciál előzés közben

$$\Phi = 2 + 3\,\varphi, \qquad \varphi = \operatorname{clip}\left(\frac{d_0 - d}{d_0 + 5}, 0, 1\right), \tag{11}$$

előzésen kívül pedig nulla. Itt d_0 a megelőzendő jármű távolsága a kihúzódás pillanatában, d pedig az aktuális távolság, így φ az elhaladás előrehaladását méri. A kihúzódáskor $\Phi = 2$, az elhaladás befejeztével $\Phi = 5$. Ennek következményei:

- a kihúzódás pillanatában $F = +1{,}96$, vagyis a bónusz értékének egy része azonnal megérkezik, nem csak mintegy öt másodperccel később;
- amíg $\Phi > 0$, lépésenként $-(1-\gamma)\Phi$ is jár, ezért az elhúzódó előzés azonnal rosszabbnak látszik;
- a visszasoroláskor $F = -5$, amit a $+5$ bónusz pontosan kiegyenlít;
- a feladott vagy ütközéssel végződő előzés visszaveszi a korábban kapott jutalmat.

A $\gamma = 0{,}98$ érték megegyezik a tanítás diszkonttényezőjével.

Megjegyzendő, hogy a formáló tag miatt az epizódonként gyűjtött, diszkontálatlan jutalomösszeg előzésenként mintegy tízzel kisebb, mint az R4 esetében. A futásokat ezért nem a jutalomösszeggel, hanem feladatszintű mérőszámokkal kell összehasonlítani, amint azt a 6. fejezet teszi.

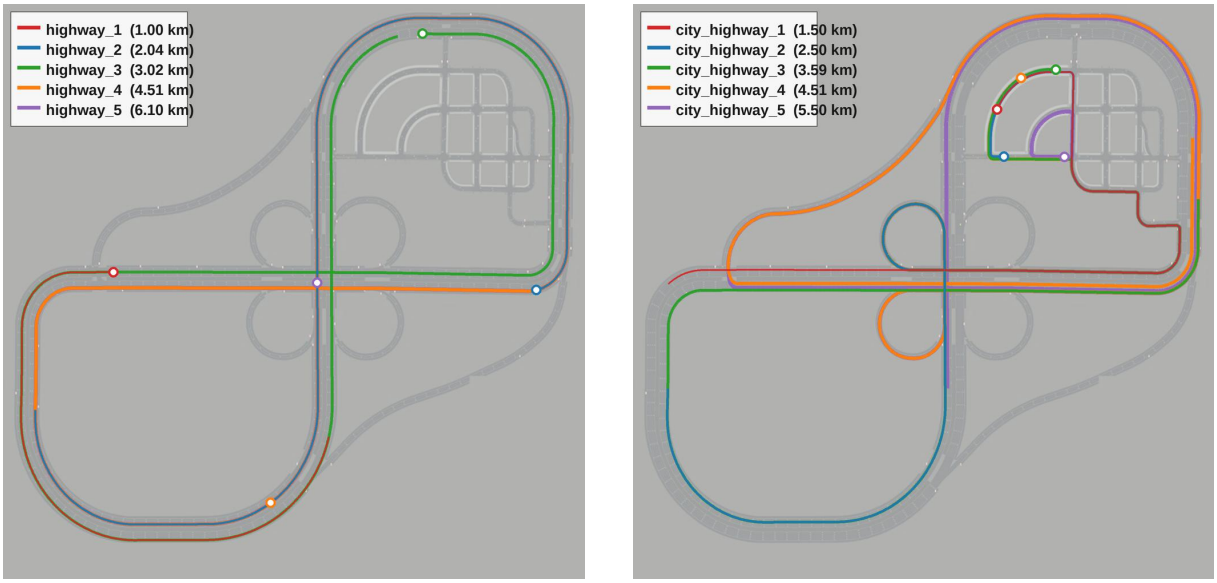
6. Elkészült modellek összehasonlítása

6.1. A kiértékelés módszertana

A megerősítéses tanulásban egyetlen futás eredménye valószínűségi változó: ugyanaz a betanított modell ugyanazon az útvonalon más eredményt ad, ha a forgalom elhelyezése megváltozik. Ezt

a saját mérésem is megerősítette. Az egyik modell ugyanazon az útvonalon az egyik forgalmi elrendezésben 32,5 km/h átlagsebességgel és két szabályos előzéssel ért célba, egy másikban 41,0 km/h-val és tízzel. A szakirodalom éppen ezért többszöri mérést és a bizonytalanság közlését várja el [28], a CARLA-alapú munkák pedig jellemzően útvonalanként több ismétlést futtatnak [29].

Ennek megfelelően a kiértékelés tíz rögzített útvonalon történt, amelyekből öt tisztán autópályás, öt pedig városi szakaszról autópályára vezet (15. ábra). Mind az öt modell mind a tíz útvonalat ötször járta be, egymástól független forgalmi elrendezésekkel, ami összesen 250 lefutást jelent.



(a) autópályás útvonalak (b) városból autópályára vezető útvonalak

15. ábra. A kiértékeléshez használt tíz útvonal a Town04 térképen. A fehér körök az indulási pontokat jelölik, az útvonalak hossza 1,05 és 6,09 km között változik.

A véletlenszám-generátor kezdőértékeit úgy választottam meg, hogy a modellek körönként pontosan ugyanazt a forgalmi helyzetet kapják: így az összehasonlítás párosított, és a különbségekben nem jelenik meg az, hogy melyik modellnek jutott könnyebb forgalom.

A kiértékelés nem áll le az első hibánál. Ha a jármű lehagyja az utat, megáll vagy ütközik, a hibát rögzítem, a járművet visszateszem az útvonal utolsó érintett pontjára, és a bejárás folytatódik. Ez a CARLA Leaderboard logikája: az útvonal végigmegy, a szabálysértések pedig számlálódnak. Így minden modell ugyanannyi lépést vezet, és a sebességre, sávtartásra, előzésre vonatkozó mérőszámok ténylegesen összevethetők.

Az útvonalak hossza 1,05 és 6,09 km között változik, ezért az eredményeket két lépésben átlagoltam. Először útvonalanként vontam össze az öt bejárást, majd a tíz útvonal átlagát képeztem. Így minden útvonal azonos súllyal szerepel, és nem a hosszabb szakaszok határozzák meg az eredményt. A táblázatokban és az ábrákon közölt szórás a tíz útvonal közötti ingadozást mutatja, vagyis azt, mennyire egyenletes a modell teljesítménye eltérő körülmények között.

Fontos megjegyzés a jutalomértékekről: mivel az öt modell eltérő jutalomfüggvénnyel tanult, az általuk gyűjtött jutalom skálája és összetétele különbözik, ezért a jutalomösszeg modellek közötti összehasonlításra nem alkalmas. Az értékelés ezért kizárólag feladatszintű, a jutalomfüggvénytől független mérőszámokra épül. Ezek a mérőszámok minden modellnél azonos módon keletkeznek, mert az előzést felismerő állapotgép mind az öt jutalomfüggvényben ugyanaz a kód.

6.2. A vizsgált jutalomváltozatok

Az összehasonlított öt jutalomváltozat részletes leírása a 5. fejezetben található; az áttekinthetőség kedvéért a köztük lévő eltéréseket a 4. táblázat foglalja össze.

4. táblázat. A jutalomváltozatok felépítése. A ✓ azt jelöli, hogy az adott tag szerepel a jutalomban.

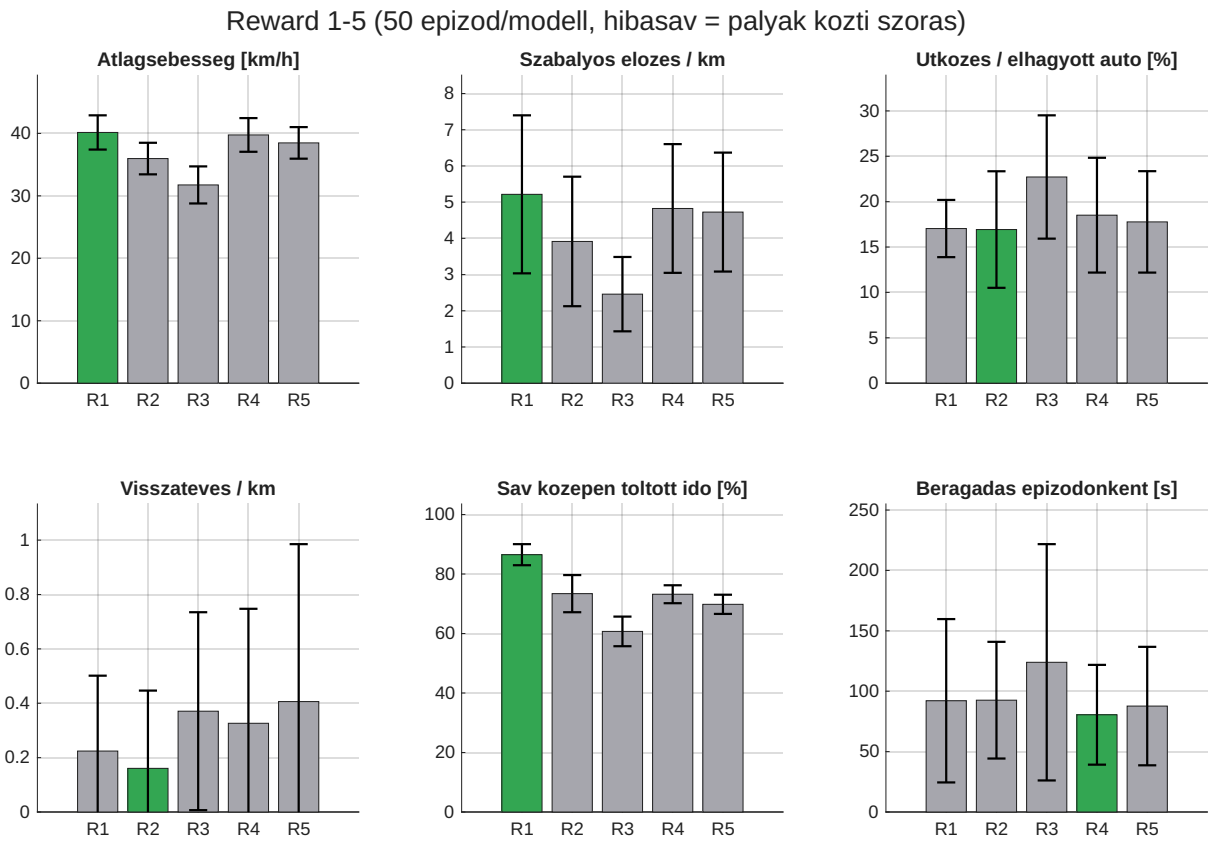
Jutalomtag	R1	R2	R3	R4	R5
Előzési állapotgép	✓	✓	✓	✓	✓
Követési távolság (headway)	✓	✓	✓	—	—
Ütközésidő-büntetés követésnél	✓	✓	✓	—	—
Ütközésidő-büntetés előzésnél	—	✓	✓	—	—
Lemaradás büntetése	25 m-en belül	25 m-en belül	teljes 40 m	—	—
Sávközép- és simasági szorzó	✓	✓	✓	—	—
Haladásalapú alapjutalom	—	—	—	✓	✓
Előzési bónusz	+5	+5	+2 és +3	+5	+5
Potenciálalapú formálás	—	—	—	—	✓

6.3. Eredmények

Az öt modell mérőszámait a 5. táblázat tartalmazza, a hozzá tartozó ábrázolás pedig a 16. ábrán látható.

5. táblázat. Az öt jutalomváltozat összehasonlítása. Minden érték a tíz útvonal átlaga, útvonalanként öt bejárásból; a ± utáni szám az útvonalak közötti szórás. A félkövér érték az adott oszlop legjobb eredménye.

	Sebesség [km/h]	Előzés [1/km]	Ütközés [%]	Visszatevés [1/km]	Sávtartás [%]	Beragadás [s]
R1	**40,2** ± 2,7	**5,22** ± 2,18	17,0 ± 3,2	0,22 ± 0,28	**86,5** ± 3,5	92,1 ± 67,6
R2	36,0 ± 2,5	3,91 ± 1,79	**16,9** ± 6,4	**0,16** ± 0,29	73,4 ± 6,2	92,6 ± 48,3
R3	31,8 ± 3,0	2,46 ± 1,03	22,7 ± 6,8	0,37 ± 0,36	60,7 ± 5,0	124,0 ± 97,8
R4	39,8 ± 2,7	4,82 ± 1,78	18,5 ± 6,3	0,33 ± 0,42	73,2 ± 3,0	**80,5** ± 41,3
R5	38,5 ± 2,5	4,73 ± 1,64	17,8 ± 5,6	0,41 ± 0,58	69,8 ± 3,2	87,7 ± 49,0



16. ábra. A hat mérőszám jutalomváltozatonként. A hibasáv az útvonalak közötti szórás, a zöld oszlop az adott mérőszám legjobb értéke.

A mérőszámok mellett az előzési kísérletek kimenetele is tanulságos (6. táblázat). Ez nem csak azt mutatja meg, hány előzés sikerült, hanem azt is, hogyan romlott el: a jármű feladta a megkezdett manővert, ütközött, vagy záróvonalon sorolt vissza.

6. táblázat. Az előzési kísérletek kimenetelének megoszlása. Az *n* oszlop a kísérletek száma mind a 250 lefutásban.

	n	Sikeres [%]	Feladta [%]	Ütközött [%]	Záróvonalon [%]
R1	440	70,5	9,5	10,0	6,4
R2	422	63,5	21,1	**7,3**	**4,5**
R3	345	46,4	25,8	13,6	7,8
R4	404	**73,0**	**7,4**	8,9	6,4
R5	437	71,6	8,7	9,2	8,0

6.4. A módosítások értékelése

R1 → R2: ütközésidő-büntetés előzhető helyzetben. A módosítás elérte a célját, de magas áron. A megkezdett, majd feladott előzések aránya 9,5%-ról 21,1%-ra nőtt, a sikeres kísérleteké 70,5%-ról 63,5%-ra csökkent, a sebesség pedig 40,2 km/h-ról 36,0 km/h-ra esett. Cserébe az ütközéssel végződő kísérletek aránya 10,0%-ról 7,3%-ra, a visszatevések száma pedig 0,22-ről 0,16-ra csökkent kilométerenként. A magyarázat a küszöbérték nagyságrendjében keresendő: a 7 s-os ütközésidő 50 és 25 km/h sebességkülönbségnél mintegy 48 m-es távolságnak felel meg, vagyis a büntetés már akkor hatott, amikor a jármű az előtte haladót egyáltalán észlelte. A modell ezért inkább lassított, mint előzött.

R2 → R3: hangolás bátrabb előzésre. Ez a változat a szándékolttal ellentétes eredményt hozott, és ez a kísérletsorozat legtanulságosabb megfigyelése. A négy módosítás célja a határozottabb előzés volt, ehhez képest az R3 minden mérőszámban a leggyengébb: 2,46 szabályos előzés kilométerenként (az R1 értékének fele), 31,8 km/h átlagsebesség, 60,7%-os sávtartás és 22,7%-os ütközési arány. Az előzési kísérletek száma is a legalacsonyabb (345 a többi modell 400 feletti értékével szemben), és ezek kevesebb mint fele sikerült.

A visszatevések okai mutatják meg, mi történt. Míg a többi modellnél a hibák túlnyomó része az útról való letérés, az R3-nál 20 esetben a sebességhatár túllépése, 31 esetben pedig a jármű megállása okozta a beavatkozást. A megemelt előzési lépésjutalom (1,1, ami meghaladja a saját sávban elérhető 1,0-t) és a kétszeres súlyú ütközésidő-büntetés együtt szélsőséges viselkedést tanított meg: a modell vagy gyorsított a megengedett sebesség fölé, vagy megállt. A beragadással töltött idő is itt a legmagasabb (124,0 s bejárásonként).

R3 → R4: áttérés haladásalapú jutalomra. A kézzel formált tagok teljes elhagyása visszanyerte az elveszített teljesítményt: 4,82 előzés kilométerenként, 39,8 km/h sebesség, és a legmagasabb

sikerarány az előzési kísérletekben (73,0%). Ez a legfontosabb eredmény a jutalomtervezés szempontjából, mert azt mutatja, hogy a követési távolságot, az ütközésidőt, a lemaradást és a sávközepet külön kezelő tagok nem javították, hanem rontották a viselkedést. Az egyetlen fizikai mennyiség, az útvonal menti haladás jutalmazása elegendőnek bizonyult. Az R4 emellett a legkevesebbet ragadt be (80,5 s).

R4 → R5: potenciálalapú formálás. A két változat gyakorlatilag egyenértékű: a mérőszámok eltérése minden esetben kisebb, mint az útvonalak közötti szórás. Ez összhangban van az elmélettel, amely szerint a potenciálalapú formálás nem változtatja meg az optimális stratégiát [27]. A formálás így ebben a feladatban nem hozott mérhető javulást a végső teljesítményben.

A sávtartás ára. Az R1 sávtartása kiemelkedik (86,5%), és ez a legmegbízhatóbb különbség az egész mérésben: az értékek szórása 3,0 és 6,2 között van, miközben a legjobb és a legrosszabb modell között 25,8 százalékpont a különbség. Az R1 jutalmában szerepel a sávközép- és a simasági szorzó, amely az R4 és R5 változatból kimaradt. A lazább sávtartás ugyanakkor nem járt több tényleges hibával: a visszatevések száma a négy változatnál egy nagyságrendbe esik.

6.5. Az eredmények megbízhatósága

Az egyes mérőszámok nem egyformán alkalmasak a modellek rangsorolására. A sebesség, a szabályos előzések száma és a sávtartás esetében a modellek közötti különbség nagyobb, mint az útvonalak közötti ingadozás, ezért ezekre megállapítás építhető. A visszatevések számánál és a beragadással töltött időnél azonban az ingadozás meghaladja a modellek közötti eltérést (0,25 ellenében 0,39, illetve 43,5 s ellenében 60,8 s), ezért ezeknél a sorrend nem tekinthető megalapozottnak. Az ütközési arány határesetet képez.

Ezt a bizonytalanságot érdemes kiemelni, mert jól szemlélteti az ismételt mérés szükségességét. Egy korábbi, útvonalanként egyszeri méréssorozat alapján az R5 tűnt a legjobb modellnek minden mérőszámban; a teljes, 250 lefutásos mérés ezt nem erősítette meg. Az egyetlen bejáráson alapuló összehasonlítás tehát félrevezető lett volna.

Végül meg kell jegyezni, hogy mindegyik jutalomváltozathoz egyetlen betanított modell tartozik. A fenti megállapítások ezért az öt konkrét modellre vonatkoznak, és nem választják szét a jutalomfüggvény hatását a tanítás véletlenszerűségétől. Ennek tisztázásához változatonként több, eltérő kezdőértékkel betanított modellre lenne szükség [28].

7. Összegzés

Hivatkozások

[1] World Health Organization. *Global Status Report on Road Safety 2023*. Geneva: World Health Organization, 2023. ISBN: 978-92-4-008651-7. URL: https://www.who.int/publications/i/item/9789240086517 (elérés dátuma 2026. 09. 29.).

[2] Steven Macenski és tsai. „Robot Operating System 2: Design, Architecture, and Uses in the Wild". *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074.

[3] Shinpei Kato és tsai. „Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems". *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. 2018, 287–296. old. DOI: 10.1109/ICCPS.2018.00035.

[4] Alexey Dosovitskiy és tsai. „CARLA: An Open Urban Driving Simulator". *Proceedings of the 1st Annual Conference on Robot Learning*. 2017, 1–16. old.

[5] CARLA Team. *CARLA: Open-source Simulator for Autonomous Driving Research*. URL: https://carla.org (elérés dátuma 2026. 10. 01.).

[6] CARLA Team. *CARLA Simulator Documentation (0.9.16)*. URL: https://carla.readthedocs.io/en/0.9.16/ (elérés dátuma 2026. 10. 01.).

[7] Dean A. Pomerleau. „ALVINN: An Autonomous Land Vehicle in a Neural Network". *Advances in Neural Information Processing Systems 1 (NIPS)*. 1989, 305–313. old.

[8] Richard S. Sutton és Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2. kiad. Cambridge, MA: MIT Press, 2018. ISBN: 978-0-262-03924-6.

[9] Volodymyr Mnih és tsai. „Human-level Control through Deep Reinforcement Learning". *Nature* 518.7540 (2015), 529–533. old. DOI: 10.1038/nature14236.

[10] John Schulman és tsai. „Proximal Policy Optimization Algorithms". *arXiv preprint arXiv:1707.06347* (2017).

[11] Timothy P. Lillicrap és tsai. „Continuous Control with Deep Reinforcement Learning". *4th International Conference on Learning Representations (ICLR)*. 2016.

[12] Tuomas Haarnoja és tsai. „Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 80. köt. Proceedings of Machine Learning Research. 2018, 1861–1870. old.

[13] Python Software Foundation. *The Python Programming Language*. URL: https://www.python.org (elérés dátuma 2026. 10. 02.).

[14] Adam Paszke és tsai. „PyTorch: An Imperative Style, High-Performance Deep Learning Library". *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, 8024–8035. old.

[15] Antonin Raffin, Jens Kober és Freek Stulp. „Smooth Exploration for Robotic Reinforcement Learning”. *Proceedings of the 5th Conference on Robot Learning (CoRL)*. 164. köt. Proceedings of Machine Learning Research. 2022, 1634–1644. old.

[16] Tuomas Haarnoja és tsai. „Soft Actor-Critic Algorithms and Applications”. *arXiv preprint arXiv:1812.05905* (2018).

[17] Antonin Raffin és tsai. „Stable-Baselines3: Reliable Reinforcement Learning Implementations”. *Journal of Machine Learning Research* 22.268 (2021), 1–8. old.

[18] Mark Towers és tsai. „Gymnasium: A Standard Interface for Reinforcement Learning Environments”. *arXiv preprint arXiv:2407.17032* (2024).

[19] Kaiming He és tsai. „Deep Residual Learning for Image Recognition”. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, 770–778. old. DOI: 10.1109/CVPR.2016.90.

[20] Karen Simonyan és Andrew Zisserman. „Very Deep Convolutional Networks for Large-Scale Image Recognition”. *3rd International Conference on Learning Representations (ICLR)*. 2015.

[21] Jiuming Liu és tsai. „TopoLiDM: Topology-Aware LiDAR Diffusion Models for Interpretable and Realistic LiDAR Point Cloud Generation”. *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2025. arXiv: 2507.22454.

[22] Yatian Pang és tsai. „Masked Autoencoders for Point Cloud Self-supervised Learning”. *Computer Vision – ECCV 2022*. 2022, 604–621. old. DOI: 10.1007/978-3-031-20086-1_35. URL: https://www.ecva.net/papers/eccv_2022/papers_ECCV/papers/136620591.pdf.

[23] Quentin Garrido és tsai. „RankMe: Assessing the Downstream Performance of Pretrained Self-Supervised Representations by Their Rank”. *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 202. köt. Proceedings of Machine Learning Research. 2023.

[24] Maxime Oquab, Timothée Darcet, Théo Moutakanni és tsai. „DINOv2: Learning Robust Visual Features without Supervision”. *Transactions on Machine Learning Research* (2024).

[25] Min Mei és A. G. R. Bullen. „Lognormal Distribution for High Traffic Flows”. *Transportation Research Record* 1398 (1993), 125–128. old.

[26] Meixin Zhu és tsai. „Safe, Efficient, and Comfortable Velocity Control based on Reinforcement Learning for Autonomous Driving”. *Transportation Research Part C: Emerging Technologies* 117 (2020), 102662. old. arXiv: 1902.00089.

[27] Andrew Y. Ng, Daishi Harada és Stuart Russell. „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping”. *Proceedings of the 16th International Conference on Machine Learning (ICML)*. 1999, 278–287. old.

[28] Rishabh Agarwal és tsai. „Deep Reinforcement Learning at the Edge of the Statistical Precipice”. *Advances in Neural Information Processing Systems (NeurIPS)*. 34. köt. 2021, 29304–29320. old. arXiv: 2108.13264.

[29] Bernhard Jaeger és tsai. „CaRL: Learning Scalable Planning Policies with Simple Rewards”. *arXiv preprint* (2025). arXiv: 2504.17838.